

CXL-RecordCache 开发文档

Chapter 3.RecordCache Design Spec v2

开发日程

开发周期:

阶段一:

主要实现功能模块:

1. CXL 内存分配, 确保从 SSD 中加载的内存页面首先被加载到 CXL 中。dax 设备映射方式。
 - a. 先写一个小的 demo 确保使用的内存是 CXL 的。以及详细打印当前物理机上本地内存 vs CXL 内存的容量大小。✅
2. Challenge 1: 后台线程负责 Promotion, 实现设计文档中的两层访问频率统计(Hot Page vs Hot Record), 以及根据页面上 Record 访问的比例确定是全页 Promote 还是选择几条 Record 做 Promote。
 - a. 中间的几个数据结构组件:
Count-Min-Sketch, Visit-Count-Histogram, Promote 线程(Promote 阶段上锁的锁结构), RecordCache Hash 表结构, Memory Slab Allocator。
3. 核心是得出可靠的准入阈值水位线。水位线根据 Zipfian 分布和 DRAM vs CXL 容量得到。
4. 可调和可配置的 DRAM RecordCache 大小以及 CXL BufferPool 大小

阶段二:

主要实现功能模块:

1. RecordCache Read-Only 版本, 完成 Promote 线程, Invalidation 线程, RCU 变种, (设计成开关模块) 集成 Lookup 和 Update 查询设计。

阶段三:

主要实现功能模块:

1. 在阶段二基础上, 支持 Write-Through 模式的功能设计。

!!大纲:

research 一共可以分为 4 个模块:

1. 支持 cxl 作为dax 设备加入原来的存储引擎层，从 dram-ssd 变成 dram-cxl-ssd (目前已经无缝集成了 dram buffer pool 和 cxl buffer pool ，并且完成了各自层级的驱逐逻辑关系) 当前分支叫做 research-prototype
2. 提供了一个筛选的算法，分为两层，第一层按 page 粒度的访问次数采样准入阈值，第二层根据当前 page 上 record 的访问比例情况，决定把当前页面全部推送进 dram buffer pool ，还是只选择该页面上的 hot record，推送到 recordCache
3. 集成 Read-Only 的recordCache，支持无锁快速读和 Update Invalidation 的高速失效。
4. 集成 Write-Through 的 recordCache，展示该设计的写性能。

Phase 1

最小交付指标

2.24

开发阶段指标:

1. CXL Allocator: 写一个小 Demo 验证内存是从 CXL 设备上分配的。
2. BufferPool 初始化的逻辑，首先加载到 CXL Memory，可配置。
3. Count-Min-Sketch 的二维数组实现(CMS 可以正确计数)，二维数组大小可配置。
4. Bucket 的老化和衰减
5. 打印当前访问频率直方图。

CXL Allocator

首先查看本地物理机的 CXL 状态:

目前的理解是，(Direct Access) 让应用程序直接访问内存的技术，绕过操作系统的页缓存 (Page Cache)。当 CXL 内存被配置为"Dax设备"(devdax), 它不会被加入到系统通用的可用内存池(free 命令看到的 System RAM)。会以字符设备的形式存在于 /dev/ 目录下，例如 (/dev/dax0.0)。

只有显式打开并映射该 /dev/dax 设备的应用程序才能使用这部分内存。应用可以直接读写这块物理内存，减少内核拷贝开销，场景是需要动态管理内存，不想让普通系统进程抢占 CXL 内存的场景。



复制代码

```
(base) [root@GNR-91 ~]# cxl list -M -u
```

```
bash: cxl: command not found...
```

```
^C
```

```
(base) [root@GNR-91 ~]# lsmem
```

RANGE	SIZE	STATE	REMOVABLE	BLOCK
-------	------	-------	-----------	-------

```

0x0000000000000000-0x000000007fffffff 2G online yes 0
0x0000000100000000-0x00000017ffffffff 1.5T online yes 2-767

Memory block size: 2G
Total online memory: 1.5T
Total offline memory: 0B
(base) [root@GNR-91 ~]# numactl -H
available: 1 nodes (0)
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46
47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70
71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94
95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113
114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131
132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149
150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167
168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185
186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203
204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221
222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239
node 0 size: 1545014 MB
node 0 free: 1531696 MB
node distances:
node 0
0: 10
(base) [root@GNR-91 ~]# free -h
              total          used          free        shared  buff/cache
available
Mem:          1.5Ti          7.4Gi          1.5Ti           83Mi          5.6Gi
              1.5Ti
Swap:         4.0Gi           0B           4.0Gi

```

目前看字符设备的列表，比较像这里的 cxlmem。

没有区分出单独的 numa 节点，240 个核都属于节点 0，

/dev/ 目录里 /dev/cxlmem 目录是 CXL 内存设备控制接口，/dev/dax0.0 是字符设备(c), DAX 设备文件。

d 表示目录，c 表示字符设备，b 表示块设备，l 表示符号链接。

```

● (base) [root@GNR-91 ~]# ls -l /dev/
total 0
drwxr-xr-x  2 root root    100 Feb  6 18:01 acl
crw-r--r--  1 root root   10, 235 Feb  6 18:01 autofs
drwxr-xr-x  2 root root   180 Feb  9 09:05 block
drwxr-xr-x  2 root root    60 Feb  7 02:01 bsg
drwxr-xr-x  3 root root    60 Feb  7 02:01 bus
drwxr-xr-x  2 root root  12300 Feb  9 09:05 char
crw-----  1 root root     5,  1 Feb  6 18:01 console
lrwxrwxrwx  1 root root     11 Feb  7 02:01 core -> /proc/kcore
drwxr-xr-x 242 root root   4840 Feb  7 02:01 cpu
crw-----  1 root root   10, 62 Feb  6 18:01 cpu_dma_latency
drwxr-xr-x  2 root root    40 Feb  6 18:05 cxlmem
crw-----  1 root root  250,  0 Feb  6 18:03 dax0.0
drwxr-xr-x  7 root root   140 Feb  7 02:01 disk
brw-rw----  1 root disk  252,  0 Feb  6 18:01 dm-0
brw-rw----  1 root disk  252,  1 Feb  6 18:01 dm-1
brw-rw----  1 root disk  252,  2 Feb  6 18:01 dm-2
drwxr-xr-x  3 root root    80 Feb  6 18:01 dri
crw-rw----  1 root video  29,  0 Feb  6 18:01 fb0
lrwxrwxrwx  1 root root    13 Feb  7 02:01 fd -> /proc/self/fd
crw-rw-rw-  1 root root    1,  7 Feb  6 18:01 full
crw-rw-rw-  1 root root  10, 229 Feb  6 18:01 fuse
crw-----  1 root root  10, 228 Feb  6 18:01 hpet
drwxr-xr-x  2 root root     0 Feb  6 18:01 hugepages
crw-----  1 root root  10, 183 Feb  6 18:01 hwrng
lrwxrwxrwx  1 root root    12 Feb  6 18:01 initctl -> /run/initctl
drwxr-xr-x  3 root root   120 Feb  6 18:01 input
crw-rw-rw-  1 root root  10, 129 Feb  6 18:01 ioasid
crw-----  1 root root 241,  0 Feb  6 18:01 ipmi0
crw-----  1 root root  10,  61 Feb  6 18:01 isst_interface
crw-r--r--  1 root root    1, 11 Feb  6 18:01 kmsg
crw-rw-rw-  1 root kvm   10, 232 Feb  6 18:01 kvm
lrwxrwxrwx  1 root root    28 Feb  7 02:01 log -> /run/systemd/journal/dev-log
crw-rw----  1 root disk  10, 237 Feb  6 18:01 loop-control
crw-rw----  1 root lp     6,  0 Feb  6 18:01 lp0
crw-rw----  1 root lp     6,  1 Feb  6 18:01 lp1
crw-rw----  1 root lp     6,  2 Feb  6 18:01 lp2
crw-rw----  1 root lp     6,  3 Feb  6 18:01 lp3
drwxr-xr-x  2 root root   120 Feb  6 18:01 mapper
crw-----  1 root root  10, 227 Feb  6 18:01 mcelog
crw-r----- 1 root kmem    1,  1 Feb  6 18:01 mem
drwxrwxrwt  2 root root    40 Feb  7 02:01 mqueue
drwxr-xr-x  2 root root    60 Feb  6 18:01 net
crw-rw-rw-  1 root root    1,  3 Feb  6 18:01 null
crw-----  1 root root  10, 144 Feb  6 18:01 nvram
crw-r----- 1 root kmem    1,  4 Feb  6 18:01 port
crw-----  1 root root  108,  0 Feb  6 18:01 ppp
crw-rw-rw-  1 root tty    5,  2 Feb 24 10:57 ptmx
drwxr-xr-x  2 root root     0 Feb  7 02:01 pts

```

通过 lspci 可以看到 cxl 设备，但是当前 /dev/cxlmem 目录下为空，不确定 cxl 映射的 dax 设备是否对应这里的 dax0.0。

```

● (base) [root@GNR-91 ~]# ls -la /dev/cxlmem/
total 0
drwxr-xr-x  2 root root   40 Feb  6 18:05 .
drwxr-xr-x 20 root root 3340 Feb  9 09:05 ..
● (base) [root@GNR-91 ~]# lspci | grep -i cxl
b9:00.0 CXL: XConn Technologies XC50256 (rev b2)
ba:00.0 CXL: XConn Technologies XC50256 (rev b2)

```


细节输出这里的 pci 设备信息:

```
(base) [root@GNR-91 ~]# lspci | grep -i cxl
b9:00.0 CXL: XConn Technologies XC50256 (rev b2)
ba:00.0 CXL: XConn Technologies XC50256 (rev b2)
(base) [root@GNR-91 ~]# lspci -vv -s b9:00.0
b9:00.0 CXL: XConn Technologies XC50256 (rev b2) (prog-if 10 [CXL Memory Device (CXL 2.x)])
Physical Slot: 21
Control: I/O- Mem- BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR+ FastB2B- DisINTx-
Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
Interrupt: pin A routed to IRQ 255
NUMA node: 0
IOMMU group: 46
Capabilities: [40] Power Management version 3
Flags: PMEClk- DSI- D1- D2- AuxCurrent=375mA PME(D0+,D1-,D2-,D3hot+,D3cold-)
Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
Capabilities: [50] MSI: Enable- Count=1/1 Maskable- 64bit+
Address: 0000000000000000 Data: 0000
Capabilities: [70] Express (v2) Endpoint, MSI 00
DevCap: MaxPayload 512 bytes, PhantFunc 0, Latency L0s <64ns, L1 <1us
ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset- SlotPowerLimit 0.000W
DevCtl: CorrErr+ NonFatalErr+ FatalErr+ UnsupReq-
RlxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop-
MaxPayload 512 bytes, MaxReadReq 512 bytes
DevSta: CorrErr+ NonFatalErr- FatalErr- UnsupReq+ AuxPwr- TransPend-
LnkCap: Port #0, Speed 32GT/s, Width x8, ASPM L1, Exit Latency L1 <64us
ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp+
LnkCtl: ASPM Disabled; RCB 64 bytes, Disabled- CommClk-
ExtSynch- ClockPM- AutWidDis- BWInt- AutBWInt-
LnkSta: Speed 32GT/s (ok), Width x8 (ok)
TrErr- Train- SlotClk+ DLActive- BWMgmt- ABWMgmt-
DevCap2: Completion Timeout: Not Supported, TimeoutDis+ NROPrPrP- LTR+
10BitTagComp+ 10BitTagReq- OBFF Not Supported, ExtFmt+ EETLPPrefix+, MaxEETLPPrefixes 4
EmergencyPowerReduction Not Supported, EmergencyPowerReductionInit-
FRS- TPHComp- ExtTPHComp-
DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis- LTR+ OBFF Disabled,
AtomicOpsCtl: ReqEn+
LnkCap2: Supported Link Speeds: 2.5-32GT/s, CrossLink- Retimer+ 2Retimers+ DRS-
LnkCtl2: Target Link Speed: 32GT/s, EnterCompliance- SpeedDis-
Transmit Margin: Normal Operating Range, EnterModifiedCompliance- ComplianceSOS-
Compliance De-emphasis: -6dB
LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete+ EqualizationPhase1+
EqualizationPhase2+ EqualizationPhase3+ LinkEqualizationRequest-
Retimer+ 2Retimers- CrosslinkRes: Upstream Port
Capabilities: [b0] MSI-X: Enable- Count=1 Masked-
Vector table: BAR=0 offset=00000000
PBA: BAR=0 offset=00000010
Capabilities: [100 v2] Advanced Error Reporting
UESSta: DLP- SDES- TLP- FCP- CmpltTO- CmpltAbt- UnxCmplt- RxOF- MalfTLP- ECRC- UnsupReq- ACSViol-
UEMsk: DLP- SDES- TLP+ FCP- CmpltTO+ CmpltAbt- UnxCmplt- RxOF- MalfTLP- ECRC+ UnsupReq+ ACSViol-
UESvrt: DLP+ SDES+ TLP+ FCP+ CmpltTO- CmpltAbt- UnxCmplt- RxOF+ MalfTLP+ ECRC- UnsupReq- ACSViol-
CESta: RxErr- BadTLP- BadDLLP- Rollover- Timeout- AdvNonFatalErr+
CEMsk: RxErr- BadTLP- BadDLLP- Rollover- Timeout- AdvNonFatalErr+
AERCap: First Error Pointer: 00, ECRGGenCap+ ECRGGenEn- ECRChkCap+ ECRChkEn-
MultHdrRecCap- MultHdrRecEn- TLPPfxPres- HdrLogCap-
HeaderLog: 00000000 00000000 00000000 00000000
Capabilities: [148 v1] Device Serial Number 1e-4a-1d-54-00-1b-00-00
```

```

Capabilities: [158 v1] Power Budgeting <?>
Capabilities: [168 v1] Secondary PCI Express
LnkCtl3: LnkEquInterruptEn- PerformEqu-
LaneErrStat: LaneErr at lane: 1
Capabilities: [188 v1] Physical Layer 16.0 GT/s <?>
Capabilities: [1b0 v1] Lane Margining at the Receiver <?>
Capabilities: [1d8 v1] Extended Capability ID 0x2a
Capabilities: [20c v1] Latency Tolerance Reporting
Max snoop latency: 40960ns
Max no snoop latency: 40960ns
Capabilities: [214 v1] L1 PM Substates
L1SubCap: PCI-PM_L1.2+ PCI-PM_L1.1+ ASPM_L1.2+ ASPM_L1.1+ L1_PM_Substates+
PortCommonModeRestoreTime=10us PortTPowerOnTime=14us
L1SubCtl1: PCI-PM_L1.2- PCI-PM_L1.1- ASPM_L1.2- ASPM_L1.1-
T_CommonMode=0us LTR1.2_Threshold=0ns
L1SubCtl2: T_PwrOn=10us
Capabilities: [268 v1] Vendor Specific Information: ID=0002 Rev=4 Len=100 <?>
Capabilities: [368 v1] Vendor Specific Information: ID=0001 Rev=1 Len=038 <?>
Capabilities: [3a0 v1] Data Link Feature <?>
Capabilities: [3ac v1] Extended Capability ID 0x2f
Capabilities: [3bc v1] Designated Vendor-Specific: Vendor=1e98 ID=0007 Rev=1 Len=20 <?>
Capabilities: [3d0 v1] Designated Vendor-Specific: Vendor=1e98 ID=0003 Rev=0 Len=40 <?>
Capabilities: [42c v1] Designated Vendor-Specific: Vendor=1e98 ID=0000 Rev=1 Len=56: CXL
CXLCap: Cache- IO+ Mem+ Mem HW Init+ HDMCount 1 Viral+
CXLctl: Cache- IO+ Mem+ Cache SF Cov 0 Cache SF Gran 0 Cache Clean- Viral-
CXLSta: Viral-

```

这里通过 `lspci -vv -s b9:00.0` 输出信息的理解:

1. prog-if 10 [CXL Memory Device(CXL 2.x)] 支持 cxl 2.0 协议, Type-3 设备显示成 CXL Memory Device。
2. 从 Capabilities 字段看出来, 这里记录的 Capabilities: [42c v1] Cache- 表示不支持 cxl.cache 协议, 但是支持 cxl.io 和 cxl.mem 协议。
3. LnkCap: Port #0, Speed 32GT/s, Width x8 表示 8 个通道, 单向的理论带宽是 64GB/s。

目前对 30.74.29.99 这台机器的了解是:

1. 240 核 CPU, 只有一个 NUMA 节点
2. lsmem 显示内存一共 1.5 T, 本地内存一共 1.5 TB。
3. 物理机上有两个 Type 3 的 cxl memory device, 不支持 cxl.cache 协议, 支持 cxl.mem 和 cxl.io, 作为两个额外的内存扩展卡。

硬件拓扑现状: 1.5 T 本地内存(System RAM) + 2 个通过 PCIe 总线连接的 CXL 内存卡(字符设备)

```

● (base) [root@GNR-91 dax0.0]# cat /sys/bus/dax/devices/dax0.0/size
0
● (base) [root@GNR-91 dax0.0]# ls /sys/bus/dax/devices/
dax0.0

```

目前看上去 dax/devices 下面只有一个 dax0.0 设备, 但是大小是 0。

动态切换 cxl 内存设备(持久内存)的使用模式,

```
daxctl reconfigure-device --mode=system-ram -f dax0.1
daxctl reconfigure-device --mode=devdax -f dax0.1
```

找不到驱动, 目前唯一的 dax 设备似乎完全没用

问题描述:

现在机器上显示可以找到两个 cxl memory device,

但是系统的 /sys/bus/dax/devices/ 下面只有一个 dax0.0 设备, 打印 size = 0

发现在对应两个 cxl pci 总线下面都没有找到驱动,

/sys/bus/pci/devices/0000:b9:00.0/driver (driver 文件夹不存在)

似乎目前两个 cxl device 没有被识别成 dax 设备, 猜测是没有对应的 cxl 驱动, 查了一下 modprobe cxl-pci 还是要升级系统?

```
(base) [root@GNR-91 ~]# lspci | grep -i cxl
b9:00.0 CXL: XConn Technologies XC50256 (rev b2)
ba:00.0 CXL: XConn Technologies XC50256 (rev b2)
(base) [root@GNR-91 ~]# ls -la /dev/cxlmem/
total 0
drwxr-xr-x 2 root root 40 Feb 6 18:05 .
drwxr-xr-x 20 root root 3340 Feb 9 09:05 ..
(base) [root@GNR-91 ~]# ls /sys/bus/dax/devices/
dax0.0
(base) [root@GNR-91 ~]# cat /sys/bus/dax/devices/dax0.0/size
0
(base) [root@GNR-91 ~]# ls /sys/bus/pci
pci/
pci_express/
(base) [root@GNR-91 ~]# ls /sys/bus/pci/devices/0000:b9:00.0
aer_dev_correctable consistent_dma_mask_bits enable local_cpus remove subsystem_vendor
aer_dev_fatal current_link_speed firmware_node max_link_speed rescan uevent
aer_dev_nonfatal current_link_width iommu max_link_width reset vendor
ari_enabled d3cold_allowed iommu_group modalias resource revision
broken_parity_status device irq msi_bus subsystem
class dma_mask_bits link numa_node subsystem
config driver_override local_cpulist power subsystem_device
(base) [root@GNR-91 ~]# ls /sys/bus/pci/devices/0000:ba:00.0
aer_dev_correctable consistent_dma_mask_bits enable local_cpus remove subsystem_vendor
aer_dev_fatal current_link_speed firmware_node max_link_speed rescan uevent
aer_dev_nonfatal current_link_width iommu max_link_width reset vendor
ari_enabled d3cold_allowed iommu_group modalias resource revision
broken_parity_status device irq msi_bus subsystem
class dma_mask_bits link numa_node subsystem
config driver_override local_cpulist power subsystem_device
(base) [root@GNR-91 ~]# ls -la /sys/bus/pci/devices/0000:b9:00.0/driver
ls: cannot access '/sys/bus/pci/devices/0000:b9:00.0/driver': No such file or directory
(base) [root@GNR-91 ~]#
```

以及原本可以配置 dax 设备或者直接内存交错使用, 但是显示:

```
(base) [root@GNR-91 ~]# ls -l /dev/dax*
crw----- 1 root root 250, 0 Feb 6 18:03 /dev/dax0.0
(base) [root@GNR-91 ~]# daxctl reconfigure-device --mode=system-ram -f dax0.0
libdaxctl: daxctl_dev_enable: dax0.0: failed to enable
error reconfiguring devices: No such device or address
reconfigured 0 devices
(base) [root@GNR-91 ~]# daxctl reconfigure-device --mode=devdax -f dax0.0
libdaxctl: daxctl_dev_enable: dax0.0: failed to enable
error reconfiguring devices: No such device or address
reconfigured 0 devices
```

看上去机器也有点崩了

```

● (base) [root@GNR-91 ~]# df -h .
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/acl-root  70G   41G   30G   59% /
● (base) [root@GNR-91 ~]# df -h --output=target,avail | sort -k2 -h -r | head -n 10
/nvme0          2.8T
/nvme1          2.1T
/tmp            755G
/sys/fs/cgroup  755G
/run            755G
/dev/shm        755G
/dev            755G
/run/user/0     151G
/home          144G
/              30G
⊗ (base) [root@GNR-91 ~]# ls /nvme0
ls: reading directory '/nvme0': Input/output error
(base) [root@GNR-91 ~]# cd /nvme
nvme0/ nvme1/
● (base) [root@GNR-91 ~]# cd /nvme0
⊗ (base) [root@GNR-91 nvme0]# ls
ls: reading directory '.': Input/output error
● (base) [root@GNR-91 nvme0]# cd /nvme1
⊗ (base) [root@GNR-91 nvme1]# ls
ls: reading directory '.': Input/output error
○ (base) [root@GNR-91 nvme1]#

```

试图 fix 问题一的一些尝试

复制代码

```

# 需要安装 cxl-cli 的包, 需要 epel 源和 crb 仓库安装对应的工具
# 在拉取 epel 源的时候, 发现服务器上配置的是 mirrors.aliyuncs.com, 内网地址
# 换一个公网的源
sudo sed -i 's/mirrors.cloud.aliyuncs.com/mirrors.aliyun.com/g'
/etc/yum.repos.d/*.repo
sudo dnf clean all
sudo dnf makecache
# 尝试重新安装 epel 源
sudo dnf install epel-release

(base) [root@GNR-91 ~]# sudo dnf install epel-release
Last metadata expiration check: 0:03:25 ago on Tue 24 Feb 2026 03:58:46
PM CST.
Error:
  Problem: problem with installed package epel-aliyuncs-release-8-
15.1.al8.noarch
    - package epel-aliyuncs-release-8-15.1.al8.noarch conflicts with epel-
release provided by epel-release-8-22.el8.noarch
    - conflicting requests
(try to add '--allowerase' to command line to replace conflicting
packages or '--skip-broken' to skip uninstallable packages or '--
nobest' to use not only best candidate packages)

```



```

# 现在检查出意见有 epel-aliyuncs-release-8-15.1.al8.noarch
# 目前不需要安装新的源，看现有源能不能把支持 cxl-cli 底层库的 crb 下载下来
# 查看当前系统实际支持的仓库名称
dnf repolist all
# cxl 相关工具的安装，尝试从源码安装和配置
# ~ 目录下只有 30 G 空间，切换到 /home 目录下进行安装和配置
cd /home
mkdir zhizhi.tyf
mkdir download_source_software && cd download_source_software
git clone https://github.com/pmem/ndctl
# 使用 meson 进行构建
meson setup build --prerfix=usr --sysconfdir=/etc --libdir=/usr/lib64
# 安装失败，原因是缺少依赖 libtraceevent
dnf search libtraceevent
# 暂时先不装 libtraceevent 和 asciidoctor，最小安装。
meson setup build --prefix=/usr --sysconfdir=/etc --libdir=/usr/lib64 \
    -Dlibtracefs=disabled \
    -Ddocs=disabled

# 这一步继续卡在了一个无关紧要的测试文件上，是 linux 系统内核版本过低 (5.10)
# 导致的
# 需要把 Linux 系统内核版本升级到 5.6+，继续在 meson setup build 的时候禁止
# 掉这个测试
meson setup build --prefix=/usr --sysconfdir=/etc --libdir=/usr/lib64 \
    -Dlibtracefs=disabled \
    -Ddocs=disabled \
    -Dfwctl=disabled
meson compile -C build

```

下面列出当前系统中已经配置和可用的软件源:

```

● (base) [root@GNR-91 ~]# dnf repolist all

```

repo id	repo name	status
alinux3-module	alinux3-module	enabled
alinux3-os	alinux3-os	enabled
alinux3-plus	alinux3-plus	enabled
alinux3-powertools	alinux3-powertools	enabled
alinux3-updates	alinux3-updates	enabled
apsara-6000	apsara-6000	enabled
epel	Extra Packages for Enterprise Linux 8 - x86_64	enabled
epel-debuginfo	Extra Packages for Enterprise Linux 8 - x86_64 - Debug	disabled
epel-modular	Extra Packages for Enterprise Linux Modular 8 - x86_64	enabled
epel-modular-debuginfo	Extra Packages for Enterprise Linux Modular 8 - x86_64 - Debug	disabled
epel-modular-source	Extra Packages for Enterprise Linux Modular 8 - x86_64 - Source	disabled
epel-source	Extra Packages for Enterprise Linux 8 - x86_64 - Source	disabled
epel-testing	Extra Packages for Enterprise Linux 8 - Testing - x86_64	disabled
epel-testing-debuginfo	Extra Packages for Enterprise Linux 8 - Testing - x86_64 - Debug	disabled
epel-testing-modular	Extra Packages for Enterprise Linux Modular 8 - Testing - x86_64	disabled
epel-testing-modular-debuginfo	Extra Packages for Enterprise Linux Modular 8 - Testing - x86_64 - Debug	disabled
epel-testing-modular-source	Extra Packages for Enterprise Linux Modular 8 - Testing - x86_64 - Source	disabled
epel-testing-source	Extra Packages for Enterprise Linux 8 - Testing - x86_64 - Source	disabled

根据命令安装一个 cxl-cli 工具，目前的源里没有这个 cxl-cli。

```

(base) [root@GNR-91 ~]# sudo dnf install ndctl daxctl cxl-cli
Last metadata expiration check: 0:14:09 ago on Tue 24 Feb 2026 03:58:46 PM CST.
Package ndctl-71.1-7.0.1.al8.x86_64 is already installed.
Package daxctl-71.1-7.0.1.al8.x86_64 is already installed.
No match for argument: cxl-cli
Error: Unable to find a match: cxl-cli
(base) [root@GNR-91 ~]# cxl --version
bash: cxl: command not found...
(base) [root@GNR-91 ~]# sudo dnf search cxl
Last metadata expiration check: 0:15:21 ago on Tue 24 Feb 2026 03:58:46 PM CST.
===== Name & Summary Matched: cxl =====
yaochi-cxl-fma.x86_64 : Fabric Manager Agent and CXL Management Tool

```

排除 libtraceevent 和一个 docs 工具之后, 使用 meson setup build 成功生成,

```

Program clear.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/clear.sh)
Program pmem-errors.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/pmem-errors.sh)
Program daxdev-errors.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/daxdev-errors.sh)
Program multi-dax.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/multi-dax.sh)
Program btt-check.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/btt-check.sh)
Program label-compat.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/label-compat.sh)
Program sector-mode.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/sector-mode.sh)
Program inject-error.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/inject-error.sh)
Program btt-errors.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/btt-errors.sh)
Program btt-pad-compat.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/btt-pad-compat.sh)
Program firmware-update.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/firmware-update.sh)
Program rescan-partitions.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/rescan-partitions.sh)
)
Program inject-smart.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/inject-smart.sh)
Program monitor.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/monitor.sh)
Program max_available_extent_ns.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/max_available_extent_ns.sh)
Program pfn-meta-errors.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/pfn-meta-errors.sh)
Program track-uuid.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/track-uuid.sh)
Program cxl-topology.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-topology.sh)
)
Program cxl-region-sysfs.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-region-sysfs.sh)
Program cxl-labels.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-labels.sh)
Program cxl-create-region.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-create-region.sh)
Program cxl-xor-region.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-xor-region.sh)
Program cxl-update-firmware.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/cxl-update-firmware.sh)
Program cxl-events.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-events.sh)
Program cxl-sanitize.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-sanitize.sh)
)
Program cxl-destroy-region.sh found: YES (/bin/bash /home/zhizhi.tyf/download_source_software/ndctl/test/cxl-destroy-region.sh)
Program cxl-qos-class.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/cxl-qos-class.sh)
Program nfit-security.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/nfit-security.sh)
Program cxl-security.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/cxl-security.sh)
Program cxl-features.sh found: YES (/home/zhizhi.tyf/download_source_software/ndctl/test/cxl-features.sh)
Run-time dependency bash-completion found: YES 2.7
Build targets in project: 35

ndctl 83

User defined options
  libdir      : /usr/lib64
  prefix      : /usr
  sysconfdir  : /etc
  docs        : disabled
  libtracefs  : disabled

Found ninja-1.8.2 at /usr/bin/ninja

```

这里会有一个 fwctl.c 的测试文件过不了, 因为其中用到的 `__struct_group` 属于更高级的 5.16+ 的内核, 这里系统内核只有 5.10。避免大面积兼容性问题, 切换到一个 ndctl 的旧版本 git checkout 77

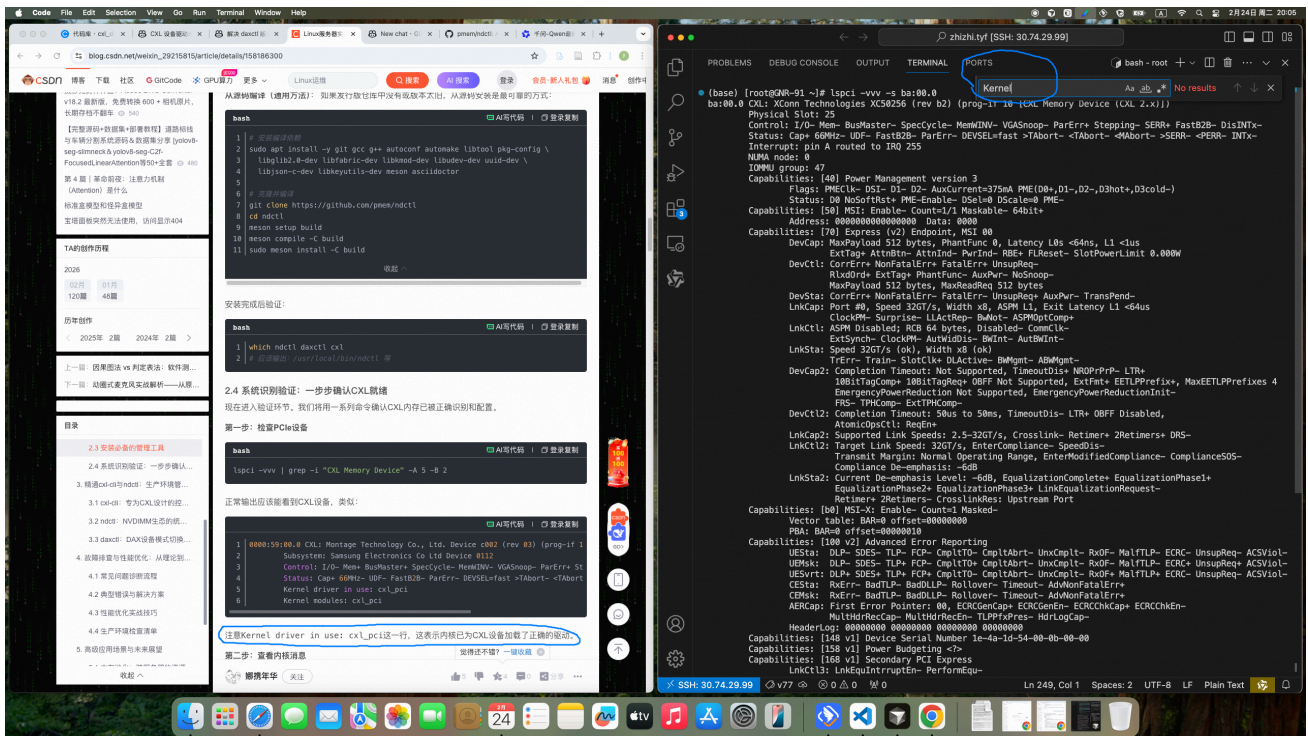
现在安装和配置好了 cxl-cli(dnf 从阿里云镜像库找不到, 但是通过源码编译已经绕过了, 直接通过 `cxl --version` 或者 `which cxl` 验证安装成功)


```

• (base) [root@GNR-91 ~]# which ndctl daxctl cxl
/usr/bin/ndctl
/usr/bin/daxctl
/usr/bin/cxl
• (base) [root@GNR-91 ~]# lspci -vvv | grep -i "CXL Memory Device"
b9:00.0 CXL: XConn Technologies XC50256 (rev b2) (prog-if 10 [CXL Memory Device (CXL 2.x)])
ba:00.0 CXL: XConn Technologies XC50256 (rev b2) (prog-if 10 [CXL Memory Device (CXL 2.x)])
• (base) [root@GNR-91 ~]# lspci | grep -i cxl
b9:00.0 CXL: XConn Technologies XC50256 (rev b2)
ba:00.0 CXL: XConn Technologies XC50256 (rev b2)

```

目前没有发现任何 Kernel driver 或者 Kernel modules 的信息



进一步准确的问题描述：

```

• (base) [root@GNR-91 ~]# lspci | grep -i "CXL"
b9:00.0 CXL: XConn Technologies XC50256 (rev b2)
ba:00.0 CXL: XConn Technologies XC50256 (rev b2)
• (base) [root@GNR-91 ~]#

```

```

(base) [root@GNR-91 ~]# lspci | grep -i cxl
b9:00.0 CXL: XConn Technologies XC50256 (rev b2)
ba:00.0 CXL: XConn Technologies XC50256 (rev b2)
(base) [root@GNR-91 ~]# ls -la /dev/cxlmem/
total 0
drwxr-xr-x 2 root root 40 Feb 6 18:05 .
drwxr-xr-x 20 root root 3340 Feb 9 09:05 ..
(base) [root@GNR-91 ~]# ls /sys/bus/dax/devices/
dax0.0
(base) [root@GNR-91 ~]# cat /sys/bus/dax/devices/dax0.0/size
0
(base) [root@GNR-91 ~]# ls /sys/bus/pci
pci/ pci_express/
(base) [root@GNR-91 ~]# ls /sys/bus/pci/devices/0000:b9:00.0
aer_dev_correctable consistent_dma_mask_bits enable local_cpus remove subsystem_vendor
aer_dev_fatal current_link_speed firmware_node max_link_speed rescan uevent
aer_dev_nonfatal current_link_width iommu max_link_width reset vendor
ari_enabled d3cold_allowed iommu_group modalias resource
broken_parity_status device irq msi_bus revision
class dma_mask_bits link numa_node subsystem
config driver_override local_cpulist power subsystem_device
(base) [root@GNR-91 ~]# ls /sys/bus/pci/devices/0000:ba:00.0
aer_dev_correctable consistent_dma_mask_bits enable local_cpus remove subsystem_vendor
aer_dev_fatal current_link_speed firmware_node max_link_speed rescan uevent
aer_dev_nonfatal current_link_width iommu max_link_width reset vendor
ari_enabled d3cold_allowed iommu_group modalias resource
broken_parity_status device irq msi_bus revision
class dma_mask_bits link numa_node subsystem
config driver_override local_cpulist power subsystem_device
(base) [root@GNR-91 ~]# ls -la /sys/bus/pci/devices/0000:b9:00.0/driver
ls: cannot access '/sys/bus/pci/devices/0000:b9:00.0/driver': No such file or directory
(base) [root@GNR-91 ~]#

```

现在的机器上有两个 CXL 设备:

通过 `lspci` 可以看到两个 CXL Memory Device, 但是发现存在如下问题:

1. `lspci -vvv -s b9:00.0` 和 `lspci -vvv -s ba:00.0` 打印细节信息时, (对比正常安装了识别 Memory Device 的设备应该看到有 `kernel driver in use: cxl_pci`), 这里没有打印出任何 `kernel driver` 或者 `kernel module` 的字段信息。
2. 没有作为单独的 `numa` 节点被识别, 打印 `numactl -H` 显示只有一个 `numa 0` 节点
3. 也没有作为 `dax` 设备被识别, 整个系统中只有一个 `dax` 字符设备, 叫做 `dax0.0`, 但是 `cat` 之后发现 `size` 为 0, (`cat /sys/bus/dax/devices/dax0.0/size`)
4. `/dev/` 目录下 `cxlmem` 目录也是空的。
5. 通过源码安装了 `cxl-cli`, (`v77` 分支版本), `cxl list -v` 的打印结果是 `Warning: no matching devices found []`
6. 使用 `daxctl` 配置得到的信息输出是: `(base) [root@GNR-91 ~]# daxctl reconfigure-device --mode=system-ram -f dax0.0`

`libdaxctl: daxctl_dev_enable: dax0.0: failed to enable`

`error reconfiguring devices: No such device or address`

`reconfigured 0 devices`, 作为 `daxdev` 也是同样的报错信息。

7. 检查 `grep -i CXL /boot/config-$(uname -r)` 无输出信息。

2.25

昨天找管理服务器的同学反馈, 得到一个 CXL FM 的文档。

(初步理解是升级了一下, 现在支持 CXL 3.1 的 Fabric Manager?)

对这里硬件拓扑的初步认识, CXL 网络 Fabric 的拓扑结构:

(1) Host node(内存的使用者, 计算节点)

和 CXL Switch 通信, 告诉 Switch 需要多少内存, 或者接收 Switch 的指令。

(2) CXL Switch mCPU

CXL 交换机本身自带的一颗管理 CPU。

现在有一个 ali_pack.tgz 安装包(包含了两个平台的程序), 需要分别传到两边去安装。

Host CPU 发出的读写指令 -> CXL Switch ASIC(纯硬件芯片转发) -> CXL Memory Device。

软件的安装阶段需要给 Switch mCPU 一个 FM 套件, 同时也需要给 Host Node 一个 FM 套件。

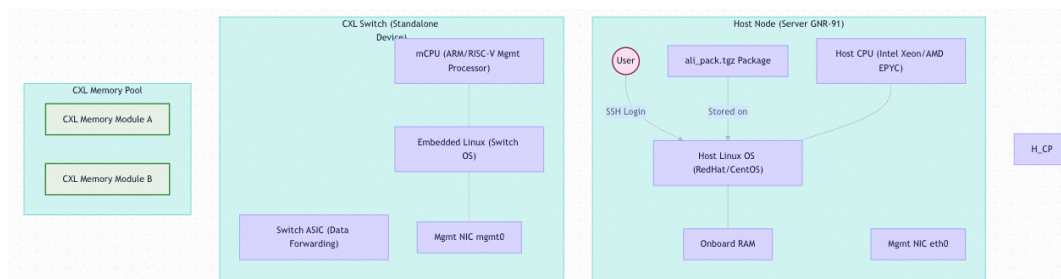
安装 CXL FM 全过程记录

安装 ali_pack-0.1.3.tgz

将安装包传输到所有的 Host node 和 CXL Switch mCPU 上。

复制代码

```
tree -L 2 ali_pack-0.1.3
# ali-fm-0.1.3 是 CXL Switch mcpu 上所需安装包, 执行该目录下的
run_install.sh, 完成在 mcpu 上的安装
# ali-fma-0.1.3 是 Host node 所需安装包, 执行该目录下的 run_install.sh, 完
成在 host node 上的安装。
```

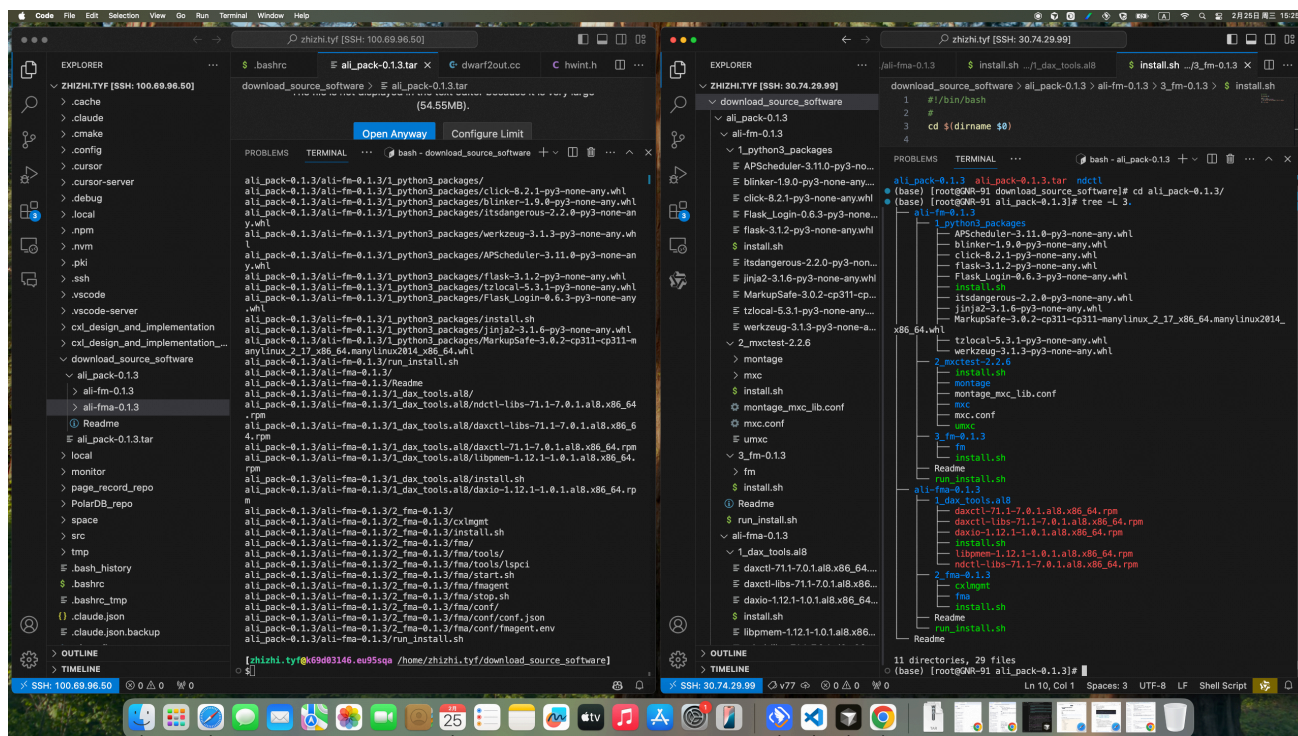


梳理整体的安装和执行流程:

CXL Switch mcpu 负责控制 CXL 交换机, Host Node 实际运行服务器节点。

1. (Install) 在管理端(CXL Switch) 和被管理端(Host Node) 分别安装对应的控制软件。
2. (mcpu Firewall)确保 CXL Switch 和 Host Node 可以互相通信: 在 mcpu 上开放 8123 TCP 端口防火墙。
3. 核心服务启动:
 - a. 启动 h3 组件, h3fm_initial 检查状态 xconn-control, 确保底层驱动就绪

- b. (start mcpu services) 启动 cxlfm 主服务 /opt/fm/start.sh, 核心的控制进程上线, 完成初始化并监听端口, 等待 Host Node 的接入。
 - c. (Power on Hosts) 开启各个 Host node 的电源。
4. (Start agent)节点配置和代理启动: 检查 Host 内核是否包含 memhp_default_state=online_movable, Host 重启完成后, 启动 fmagent, /opt/fma/start.sh



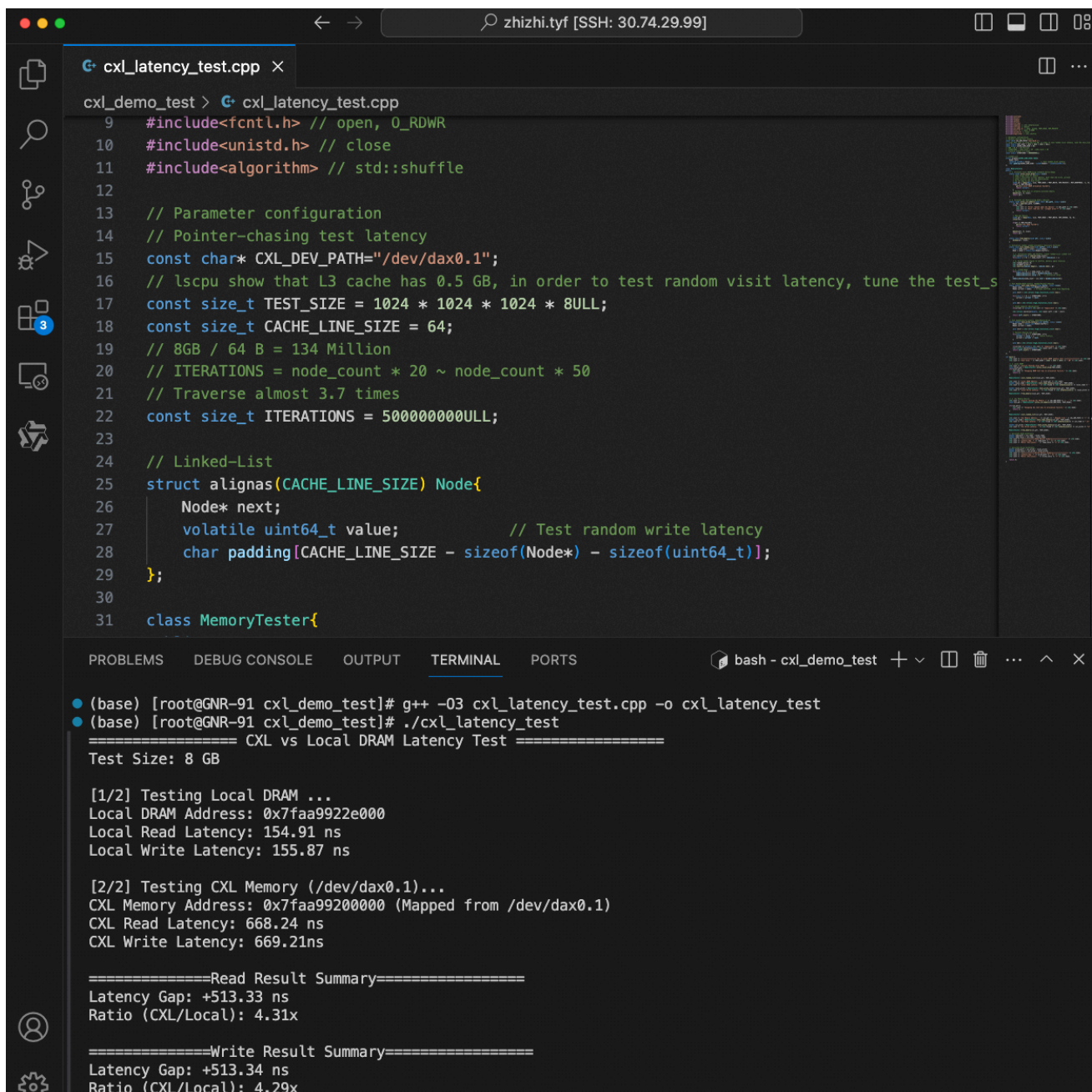
理解: 左边这台机器是 Host Node, 右边这台机器是 CXL Switch, 分别都安装阿里自研的驱动套件 fm 和 fma(❌)

测试结果的几个问题:

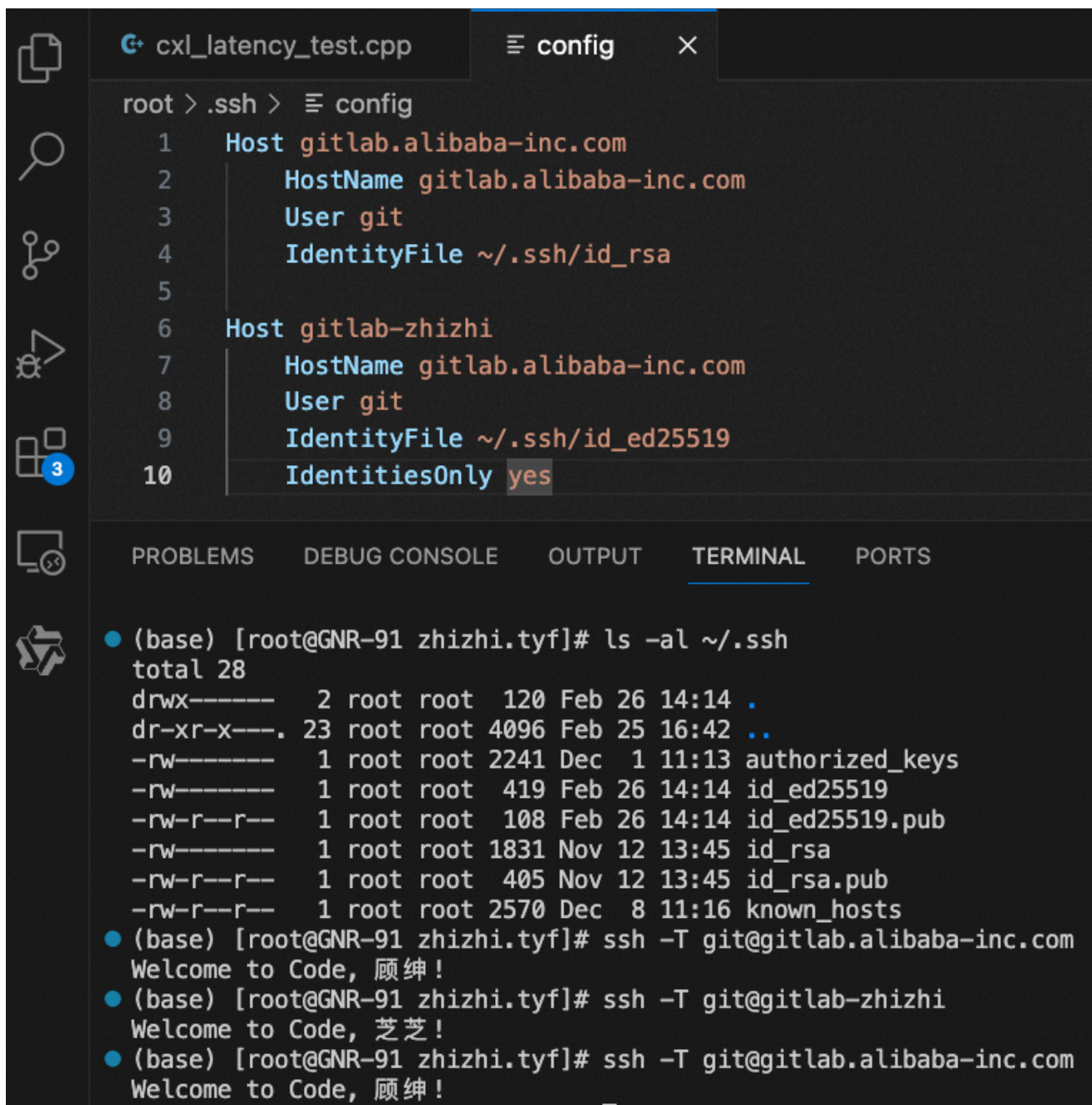
1. L3 Cache 本机是 0.5 GB, 测试采用 8 GB 链表, 每个节点填充到 64 B, 一共有约 1.34 亿个节点, 循环遍历随机访问 37 次。
2. 本地内存访问的时间是: 142.78ns, cxi 目前已经跑了 50min 还没有出结果。
3. Pointer-Swizzling 似乎用不上了, 这里 mmap 打印地址几乎没有区别, 无法从高位区分。

2.26

准备跑一个随机访问的测试, 涉及到随机读和随机写。



```
zhizhi.tyf [SSH: 30.74.29.99]
cxl_latency_test.cpp x
cxl_demo_test > G- cxl_latency_test.cpp
9  #include<fcntl.h> // open, O_RDWR
10 #include<unistd.h> // close
11 #include<algorithm> // std::shuffle
12
13 // Parameter configuration
14 // Pointer-chasing test latency
15 const char* CXL_DEV_PATH="/dev/dax0.1";
16 // lscpu show that L3 cache has 0.5 GB, in order to test random visit latency, tune the test_s
17 const size_t TEST_SIZE = 1024 * 1024 * 1024 * 8ULL;
18 const size_t CACHE_LINE_SIZE = 64;
19 // 8GB / 64 B = 134 Million
20 // ITERATIONS = node_count * 20 ~ node_count * 50
21 // Traverse almost 3.7 times
22 const size_t ITERATIONS = 500000000ULL;
23
24 // Linked-List
25 struct alignas(CACHE_LINE_SIZE) Node{
26     Node* next;
27     volatile uint64_t value; // Test random write latency
28     char padding[CACHE_LINE_SIZE - sizeof(Node*) - sizeof(uint64_t)];
29 };
30
31 class MemoryTester{
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
26
```

The screenshot shows a VS Code editor with a file named `cxl_latency_test.cpp` open. The editor is displaying an SSH configuration file (`config`) with the following content:

```
root > .ssh > config
1 Host gitlab.alibaba-inc.com
2   HostName gitlab.alibaba-inc.com
3   User git
4   IdentityFile ~/.ssh/id_rsa
5
6 Host gitlab-zhizhi
7   HostName gitlab.alibaba-inc.com
8   User git
9   IdentityFile ~/.ssh/id_ed25519
10  IdentitiesOnly yes
```

Below the editor, the **TERMINAL** tab is active, showing the following commands and output:

```
(base) [root@GNR-91 zhizhi.tyf]# ls -al ~/.ssh
total 28
drwx----- 2 root root 120 Feb 26 14:14 .
dr-xr-x---. 23 root root 4096 Feb 25 16:42 ..
-rw----- 1 root root 2241 Dec  1 11:13 authorized_keys
-rw----- 1 root root 419 Feb 26 14:14 id_ed25519
-rw-r--r-- 1 root root 108 Feb 26 14:14 id_ed25519.pub
-rw----- 1 root root 1831 Nov 12 13:45 id_rsa
-rw-r--r-- 1 root root 405 Nov 12 13:45 id_rsa.pub
-rw-r--r-- 1 root root 2570 Dec  8 11:16 known_hosts
(base) [root@GNR-91 zhizhi.tyf]# ssh -T git@gitlab.alibaba-inc.com
Welcome to Code, 顾绅!
(base) [root@GNR-91 zhizhi.tyf]# ssh -T git@gitlab-zhizhi
Welcome to Code, 芝芝!
(base) [root@GNR-91 zhizhi.tyf]# ssh -T git@gitlab.alibaba-inc.com
Welcome to Code, 顾绅!
```

这次大概花了 3h 重新把 leanstore 的配置完成了。(主要工作是折腾了一下 gcc-13.2.0, 手动换源下载依赖, 和手动安装了一些其他用到的对比软件, rocksdb, wiredtiger, lmdb, liburing, 相比第一次安装快了很多, (有时间可以折腾一下打包成一个容器镜像, 这里先不整了, 目前环境问题已经解决了))。

✓ 避免反复多次配环境

复制代码

- # 1. 基础镜像: 选择和你当前环境接近的系统 (比如 Aliyun Linux 3 对应 CentOS 8/Stream 8, 或者直接用 Ubuntu 22.04 省事)
- # 这里推荐用 Ubuntu 22.04, 因为它自带 GCC 11, 且安装依赖包比 CentOS 方便太多
- FROM ubuntu:22.04 AS builder
- # 2. 避免交互式安装卡住
- ENV DEBIAN_FRONTEND=noninteractive

```
# 3. 安装系统基础工具和依赖库
# 直接安装较新的 GCC (Ubuntu 22.04 自带 GCC 11, 如果必须 13 可以通过 ppa 装)
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    git \
    wget \
    curl \
    libbz2-dev \
    liblz4-dev \
    libsnappy-dev \
    libzstd-dev \
    liburing-dev \
    libgflags-dev \
    zlib1g-dev \
    && rm -rf /var/lib/apt/lists/*

# 4. 编译安装 RocksDB (这一步最耗时, 放在镜像里构建一次就好)
WORKDIR /tmp
RUN git clone --depth 1 --branch v8.10.0
https://github.com/facebook/rocksdb.git && \
    cd rocksdb && \
    mkdir build && cd build && \
    cmake .. -DCMAKE_BUILD_TYPE=Release -DWITH_TESTS=OFF -
DWITH_BENCHMARK_TOOLS=OFF -DWITH_TOOLS=OFF -DPORTABLE=ON && \
    make -j$(nproc) && make install && \
    cd /tmp && rm -rf rocksdb

# 5. 编译安装 WiredTiger
RUN wget https://source.wiredtiger.com/releases/wiredtiger-
11.2.0.tar.bz2 && \
    tar -jxvf wiredtiger-11.2.0.tar.bz2 && \
    cd wiredtiger-11.2.0 && \
    mkdir build && cd build && \
    cmake .. -DCMAKE_BUILD_TYPE=Release -DENABLE_PYTHON=OFF && \
    make -j$(nproc) && make install && \
    cd /tmp && rm -rf wiredtiger-11.2.0

# 6. 编译安装 LMDB
RUN git clone https://github.com/LMDB/lmdb.git && \
    cd lmdb/libraries/liblmdb && \
    make -j$(nproc) && make install && \
    cd /tmp && rm -rf lmdb

# --- 开发环境镜像 ---
```

```

# 上面是构建层，这里开始是最终你要用的镜像
FROM ubuntu:22.04

# 再次安装运行时需要的库 (runtime libraries)
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    git \
    gdb \
    vim \
    libbz2-1.0 \
    liblz4-1 \
    libsnappy1v5 \
    libzstd1 \
    liburing2 \
    && rm -rf /var/lib/apt/lists/*

# 从 builder 阶段把编译好的库复制过来
COPY --from=builder /usr/local/lib /usr/local/lib
COPY --from=builder /usr/local/include /usr/local/include
COPY --from=builder /usr/local/bin /usr/local/bin

# 刷新动态库缓存
RUN ldconfig

# 设置工作目录
WORKDIR /app

# 默认进入 bash

```

功能模块

此模块记录我们开发过程中所有支持的功能。

内存加载部分的核心功能

需求:

1. 可配置的 cxl tiering 开关

在 working set 能够完全 fit in dram 或者 dram 足够用的时候，可以不开启 cxl 开关。回退 dram-ssd 两层，开关打开后进入 dram-cxl-ssd 三层【保留这个功能的目的: 测试新加入缓存层和未增加之前的效果】

2. 三个可以调整的参数: dram_buffer_pool, cxl_buffer_pool, dram_recordcache 大小的设置, 动态调整比例

3. 修改本次工作的核心加载流程,

第一次加载从 SSD 中载入到 cxl_buffer_pool,

这里尽量不要侵入式地修改太多关于 BufferFrame 的字段逻辑,

因为本身 LeanStore 的 Pointer-Swizzling 机制是很好的, (减少了页表查找),

我们的 cxl memory device 在 mmap 的时候在虚拟地址上确实是和 dram_address 没有区别,

(这里我还在考虑要不要改这个 Swip, 让 union {page_id, dram_address, cxl_address}), 在最开始的时候分配连续的大页面, 比如 dram_address_ptr 一部分, cxl_address_ptr 一部分。

4. cxl buffer pool 的 free_list 和 dram buffer pool 的 free_list 的分配逻辑一致,

留到第二部分考虑 RecordCache 自己用 SIEVE 的驱逐和自己定义 Slab Allocator 的分配器。

这里只先做最基础的加载流程走通, `ssd -> cxl_buffer_pool -> dram_buffer_pool`

5. 第一部分实现先不考虑 dram_recordCache, 先处理

`[dram_buffer_pool] <-> [cxl_buffer_pool] <-> [ssd]` 这一套逻辑走通, 驱逐的时候 cxl_buffer_pool 直接驱逐到 ssd, dram_buffer_pool 的部分先驱逐到 cxl_buffer_pool。

6. 在正式进入修改 dram_recordcache 之前, 需要处理好 dram_buffer_pool 和 cxl_buffer_pool 之间 page migration 的逻辑。

7. 关键问题, 在加载页面的时候, 还不清楚是否是叶子页面, 但是我们的设计里, 只要碰到索引页面, 就直接不做任何 Promote 判断, 无脑进入 DRAM 层。

(目前一个不太好的实现方案是: 先加载到 cxl page 上, 然后读出 header 字段为 is_leaf == false, 再 free 掉, 再加载到 dram page 上。这里感觉会带来一定的延迟)

8. 关于分配内存的时候, 原来的 dram 分配方法是 `madvise(MADV_HUGEPAGE)` 和 `madvise(MADV_DONTFORK)`, 暂时 cxl 也这么用, 也看到有的工作对 `MADV_HUGEPAGE` 这里也进行细节修改。这里先不做优化, 按照 dram 分配的设置来。

9. 内存分配碎片的问题, 因为本身希望一次分配, 两处使用, 但是 LeanStore 原生的实现里增加了额外的 safety pages 来防止边界溢出导致程序崩溃, 如果用这个指针, 会导致对不齐的问题。

10. 设计上的问题, LeanStore 后台有一个 PageProviderThread 负责驱逐的过程, PageProviderThread 里会调用 getPartitionID, 原先直接用 dram 的话, 直接 & dram_partition_mask(0011 1111) 就可以了, 现在的核心功能需要把 PageProviderThread 拆分成两个,

对应我们方案里的两级别驱逐, dram_buffer_pool -> cxl_buffer_pool, cxl_buffer_pool -> ssd 考虑这里怎么修改这个 PageProviderThread。

PageProviderThread 也需要分成两层。

11. 设计上的问题, 关于 partitions 的路由逻辑, 原先只有一个 dram_buffer_pool 的时候, 路由的过程相对简单, 直接 pid & partitions_mask 得到, 但是现在设计的 cxl_buffer_pool 的 partitions 如果是 128, 那么当前 64 + 128 并不是一个 2 的倍数, 路由逻辑需要重新设计。
12. 整体的加载逻辑, CXL -> DRAM 热升级, DRAM -> CXL 冷降级, CXL -> SSD 驱逐, 三部分执行流程。
13. 一个值得注意的历史包袱, 由于当时 LeanStore 作者实现的时候, 把分配 Page_id 和 dram_free_list 这两个相互独立的功能写在一个数据结构 Partition 里了, 而绝大多数情况下需要立刻分配 page_id 出现在初次加载数据和页面分裂的时候, 这里需要执行立刻写入。我们认为这个功能只需要在 DRAM 层处理就可以了, 不需要 CXL 层。因此涉及到 page_id 的 serialize 和 deserialize 的逻辑只会走 DRAM 处的 Partition 对应模块。(这个历史包袱我们先不拆分了, 还是沿用, 但实际 CXL 不会用到这里的 io_ht 和 next_pid 功能)
14. 从 cxl_free_list.tryPop() cxl_buffer_frame 到拷贝到 dram_buffer_frame 的过程中, 必须要加锁, 由于之前一步的复制过程中已经复制到了 io_ht 里, pid -> IOFrame -> BufferFrame, cxl_buffer_frame 对其他线程可见。

LeanStore 原本的实现

LeanStore 的实现在内存加载这里很简单, 核心结构是 BufferManager, 有一个连续的 BufferFrame* bfs 数组, 其中原本只有 dram_buffer_pool 的写法就是 mmap 一段连续的内存, 通过 reinterpret_cast 转换成 BufferFrame* 的格式, 同时为了 Buffer Pool 申请 free Page 时, 不会因为同时抢同一个 free_list 上的锁导致阻塞, 所以做了并行的 Partition 操作。核心是 BufferManager 初始化操作, 以及之后的 resolveSwip, 如果确认当前 Swip 表示的是 page_id 就执行加载, 从对应的 Partition free-list 里 Pop 出 free Page。

涉及到的改动文件和改动文件夹

配置文件增加开关,size, cxl_tiering, cxl_dax_device_path

Config.hpp

Config.hpp

内存分配模块, bufferManager 初始化

buffer-manager/BufferManager.cpp

初始化三个内存指针, 以及合并 cxl bp partitions 到 dram bp partitions, 两者合并到一个数组上, 物理上紧挨着, 逻辑上分成不同的部分。

修改的逻辑包括使用到 `partitions_count` 的函数，比如 `BufferManager::consumedPages()` 和 `BufferManager::serialize()` 这两个功能。

核心在于对 `Partition` 的理解和修改逻辑。

目前提到需要修改的文件

- `BufferManager.hpp/cpp`, `resolveSwip` 的逻辑需要修改，从纯 DRAM 改成 CXL

(💡) 当前的设计决策，由于 DRAM 和 CXL 的分配实际上来自于两次 `mmap`，地址范围本身不重叠。Hot-Cold 的那套驱逐逻辑还适用。唯一需要改的地方是 SSD load 部分的逻辑，需要首先 load 到 CXL，然后根据是否是索引页的情况判断直接进入 DRAM。

`resolveSwip` -----> worker thread

只修改 EVICTED: SSD -> CXL(新 Page 落在 CXL)

修改后的逻辑: 从 CXL `free_list` 里拿一个空的 `BufferFrame`，`readPageSync(pid, cxl_buffer_frame.page)`, `pread()` 把 SSD 中 `pid` 对应的 16 KB 原样复制到 `cxl_buffer_frame.page`,

这里 CXL 的一个优势在于可以字节粒度寻址，所以在 `pread` 把 16 KB 的内容原样复制进 `cxl_buffer_frame.page` 的时候，`cxl_buffer_frame.page.data_id` 可读，`cxl_buffer_frame.page.dt[]` 的内容也存在。

相当于整个 `BufferFrame`, Header 的 512 B 不需要持久化, Page 的 16 KB 需要持久化。

页面布局是自描述的。

检查是否是 `InnerNode`, `BTreeNode *node = reinterpret_cast<BTreeNode*>(cxl_bf.page.dt);`

`node -> is_leaf == true.`

【这里由于 `LeanStore` 写的比较初级，所以只注册了两种类型，正常的逻辑应该是判断属于哪种类型，然后调用当前类型中注册的 `should_directly_promote_to_dram` 的判断逻辑，在判断逻辑里封装如果是 `BTreeNode`, 调用 `isInner`】

`Swip` 是 EVICTED

从 CXL `free_list` 拿一个空帧 (叫 `cxl_bf`)

`readPageSync(pid, cxl_bf.page)`

→ `pread()` 把 SSD 上的 16KB 原样复制进 `cxl_bf.page`

检查: 是 `BTree inner node` 吗?

`BTreeNode* node = reinterpret_cast<BTreeNode*>(cxl_bf.page.dt);`

→ `node->is_leaf == false` → 是 inner node, 必须住 DRAM

从 DRAM free_list 再拿一个空帧 (叫 dram_bf)

memcpy(dram_bf.page, cxl_bf.page, PAGE_SIZE)

→ 把 CXL 帧里的内容整页复制到 DRAM 帧

cxl_bf.reset()

cxl_free_list.push(cxl_bf) ← CXL 帧归还, 白借了一次

swip_value.warm(&dram_bf) ← Swip 指向 DRAM 里的帧, HOT

return dram_bf ← worker 拿到这个页, page 住在 DRAM 里

PageProviderThread -----> (Background thread)

DRAM 冷页 -> CXL(cool/evict 改成 demote 到 CXL)

CXL 热页 -> DRAM(根据我的两阶段提交 CMS 算法确定全页提升的逻辑)

CXL 冷页 -> SSD(实际执行淘汰)

- PageProviderThread.cpp 还是纯 DRAM 的单层驱逐逻辑



1. DTRegistry.hpp: 增加 should_directly_promote_to_DRAM 可选回调 + requiresDRAM() 声明
2. DTRegistry.cpp: 实现 requiresDRAM()
3. BufferManager.hpp: 增加 isInCXL/isInDRAM helper 新方法声明 + FreeBfsBatch 扩展
4. BufferManager.cpp: 修改 resolveSwip(SSD -> CXL 加载 + inner node 迁移到 DRAM)
5. PageProviderThread.cpp 拆分成两层驱逐(DRAM -> CXL, CXL -> SSD)
6. BTreeLL.cpp: 注册 should_directly_promote_to_DRAM 回调
7. BTreeVL.cpp: 注册 should_directly_promote_to_DRAM 回调

188 Loc

路由逻辑需要修正

1. reclaimPage(), 固定 push 到 partition.dram_free_list, 需要改
2. evictLastPage(), 额外处理这部分逻辑
3. getContainingBufferFrame() 暂时不支持 CXL
4. consumedPages() 遍历分区的问题
5. writeAllBufferFrames() 刷脏问题
6. pageProviderThread evict_bf lambda

7. pageProviderThread evict 的逻辑

待实现的功能: CXL -> SSD 驱逐, DRAM -> CXL 冷降级, (CXL -> DRAM) 的热升级等到第二阶段再实现,

PageProviderThread 拆分成两个层次

现有函数的 CXL 适配,

PageProviderThread 逻辑修正

DRAM -> CXL, CXL -> SSD

1. 这里分出两个页面回收线程, 分别叫做(原来的名字), 和 cxl_原来的名字

所以也需要单独维护两个 free_bfs_batch 和 free_cxl_bfs_batch

2. 对于驱逐 DRAM 中的页面, 如果发现是脏页, 这里直接降级到 CXL Buffer Pool, 回收 DRAM 所属的页面, 这个过程中可能需要锁和并发控制。
3. CXL 中的驱逐过程需要保证刷脏完成之后才能写到 SSD。
4. 这里降级的过程中不区分是叶子页面还是索引页面, 因为本身跑 CXL Buffer Pool 也需要很大的数据量, 之前设计的过程中, 加载已经被驱逐的页面, 直接从 SSD 提升到 DRAM(CXL 加载过程中 load Page) 是因为索引页不走我这里两次判断 RecordCache 的准入逻辑, 但是驱逐的时候一视同仁。
5. io_ht 这里认为就是页面并发访问控制的一个数据结构, 目前就先放在 DRAM 里, 不改这里的路由逻辑了。
6. 复杂的点在于这里 DRAM 降级过程中的并发控制, 这个操作涉及到(获取 CXL 空闲的 BufferFrame, memcpy 页面数据, 以及更新父节点的 Swip, 从 DRAM BufferFrame 指向 CXL BufferFrame, 同时回收 DRAM BufferFrame, 这个过程中需要保证没有线程正在读/写这个页面, 以及父节点 Swip 的原子切换

这里需要同时持有三把锁, 父亲节点的独占锁, 当前 BufferFrame 的独占锁, CXL BufferFrame 的初始化控制)。

整体流程:

PageProviderThread

- Cooling: 选取适合的 Hot DRAM BufferFrame 做 Cooling (合适指的是尽量避免驱逐父节点导致的写开销) Hot bit -> Cool bit
 - Eviction: 已经是 Cool 的加入 eviction_bfs 队列, 需要判断当前页面是否为脏页, 如果是脏页, 写入 AIO(Asynchronous IO)的部分做批量写回, 干净的页面直接刷脏。
 - 新增的逻辑(DRAM -> CXL, CXL -> SSD) 其中 DRAM 的部分不需要刷脏, 统一等到驱逐到 SSD 再刷脏。
7. 目前 cooling 的过程中, 索引页永远保持 Hot, 这里认为索引占整体数据的大小不变, 一般不会撑爆。(如果发现确实撑爆了, 加一层告警信息)。

8. 直接无缝衔接到原来的系统设计上。如果是 PageProviderThread 和 CXLPageProviderThread 这两个工作的线程，处理的逻辑是这样的: (1) DRAM 有 Hot 到 Cool 的 Cooling 过程, (2) CXL 有 Hot 到 Cool 的 Cooling 过程, Cool DRAM 降级到 CXL 的时候, 如果不增加标志位, 可能还是需要把 Cool DRAM 变成 Hot CXL(Swip 的语义几乎不变, CXL 在代码层面无缝衔接)。
9. 一个值得注意的历史包袱, 由于当时 LeanStore 作者实现的时候, 把分配 Page_id 和 dram_free_list 这两个相互独立的功能写在一个数据结构 Partition 里了, 而绝大多数情况下需要立刻分配 page_id 出现在初次加载数据和页面分裂的时候, 这里需要执行立刻写入。我们认为这个功能只需要在 DRAM 层处理就可以了, 不需要 CXL 层。因此涉及到 page_id 的 serialize 和 deserialize 的逻辑只会走 DRAM 处的 Partition 对应模块。

(这个历史包袱我们先不拆分了, 还是沿用, 但实际 CXL 不会用到这里的 io_ht 和 next_pid 功能)

对应的 page_id 到 DRAM BufferPool 还是 CXL BufferPool 的路由逻辑实际上无法由 page_id 得到, page_id 本身不携带层级信息。

10. 目前与 PageProviderThread 的新增分层功能有冲突的函数:

- a. reclaimPage: 页面回收的逻辑(好消息是 PageProviderThread 不走这里) ✓
- b. writeAllBufferFrames: 全量刷脏逻辑 ✓
- c. consumedPages: 缺少新的统计(本身是分配 page_id 的逻辑, 这里只需要改掉原来这个令人迷惑的名字就行, 统计已经分配出去但是还没有回收的 page 信息) ✓

LeanStore 关于 PageProviderThread 的逻辑(323Loc)

冷却状态的逻辑:

cool_candidate_bfs 队列

↓

取出一个 BF (r_buffer)

↓

检查基本条件:

keep_in_memory? is_being_written_back? isExclusivelyLatched?

↓ 都不是

已经是 COOL? → 直接进 evict_candidate_bfs

↓ 不是

不是 HOT? → 跳过

↓ 是 HOT

遍历子节点:

有 HOT 子节点? → 先处理子节点 (push进队列), 跳过当前BF

有 COOL 子节点? → 跳过当前BF (子节点还没完全驱逐)

全部 EVICTED? → 可以处理当前BF

↓
找父节点 findParent()
↓
Space Check 通过?
↓ 通过
加锁(父节点 + 当前BF)
↓
state: HOT → COOL
swip.cool(): 内存指针 → PID
↓
failed_attempts = 0 // 成功, 重置失败计数

resolveSwip 逻辑修复

▽ 复制代码

```
BufferFrame& bf = load_into_cxl ?  
randomCXLPartition().cxl_free_list.tryPop():  
randomPartition().dram_free_list.tryPop();
```

初始化时 pid = 9999

开始注册 io_ht 和读盘

▽ 复制代码

```
IOFrame& io_frame = partition.io_ht.insert(pid);  
bf.header.latch.assertNotExclusivelyLatched();  
io_frame.state = IOFrame::STATE::READING;
```

此时注册的逻辑是,io_frame 的状态还是 READING, 所以 BufferFrame* bf = nullptr;(目前没
有路径可达)

类别	总行数 (含注释)	纯代码行数 (去掉注释和空行)
已修改文件 (15 个)	702	399
新增 PageProvider 文件 (去重后)	434	321
CXL-Tiering 胶水代码总计	1136	720

测试过程中的命令

```

./build/frontend/ycsb --ycsb_read_ratio=100 --target_gib=2 --
worker_threads=4 --pp_threads=2 --dram_gib=1 --trunc --contention_split
--xmerge --print_tx_console --run_for_seconds=30 --isolation_level=si

./build/frontend/minimal_example --trunc --dram_gib=1 --
worker_threads=4 --run_for_seconds=5

```

核心是:

1. 复用了 LeanStore 对于 Buffer Pool 设计的 partition 分片,

物理上有三大块初始化的内存,

bfs = mmap(DRAM),

cxl_bfs = mmap(DaxDevice),

record_cache_ptr = mmap(DRAM),

属于三部分互相独立的内存,

然后默认情况下 DRAM BufferFrame 对应的走

partition[0..63], CXL BufferFrame 对应的走 partition[64..191],

逻辑上看是一个数组, 物理上是隔离的区域。

2. 改了原来的 PageProviderThread(页面驱逐的逻辑, 原来的写法用了很多 lambda 表达式, 把 3 个阶段的逻辑全写在一个函数里了,

(1) 采样冷却(Hot -> Cool)

(2) 执行驱逐 (Cool -> Evict)

(3) 异步刷脏

这里我拆分成了 DRAMPageProviderThread 和 CXLPAGEProviderThread,

前一个复用阶段 (1),(2),

后一个复用阶段 (1)(2)(3),

刷脏的部分统一留到 CXL 驱逐到 SSD 的时候处理。

3. 其他关于 SSD 的 page_id 分配和回收的逻辑就放在 DRAM 层, (原来代码把 page_id 的分配和 Partition 分片绑定在一个结构里了)

没有做处理修改, 只是把原来写的有歧义的 freePage 的名称改掉了。

4. BufferManager 处理页表查询的页面加载的逻辑,

这里有一个稍微复杂的，就是注册了判断当前 Page 是否是 BTreeNode 索引页的逻辑，如果是索引页，不走 record promote, 直接拷贝到 DRAM。

流程是：

(1) 分配 cxl_free_list BufferFrame

(2) 注册到 io_hash_table 状态为 reading, 表示当前 page_id 对应的页面正在加载过程中，

(3) readPageSync, 先写页面到 cxl BufferFrame, 读出 BufferFrame Header, 判断是索引页面

(4) 分配 dram_free_list BufferFrame

(5) 执行 memcpy, 并回收 cxl BufferFrame, cxl BufferFrame reset

(6) 标记 io_hash_table 状态为 ready

5. DRAM 驱逐这里沿用原来的逻辑，

没有区分索引页和数据页。

(不过原来驱逐逻辑会首先判断一下当前 Page 是否还有可以被驱逐的子节点，如果有，就优先驱逐子节点)

这里在原来逻辑基础上加了一个要求，必须是同一个层次的子节点。

6. 原先首次加载页面的逻辑没有改，allocatePage() 默认新页都是在 DRAM Tier 生成。

💡 (TODO)历史遗留问题:

关于 DRAMPageProvider 的驱逐逻辑，

原先 PageProviderThread 的逻辑是，

一旦发现父亲为 Hot,找是否有同为 Hot 的子节点替代。

需要从 Hot -> Cool, 此时因为如果优先驱逐父节点，那么对应的子页面需要进行调整。

但在跨层之后，父节点位于 DRAM 中，大概率子节点位于 CXL 中。

原先的设计想法是：

DRAM -> CXL, CXL -> SSD,

DRAM -> CXL, cool_dram_candidate_bfs 和 demotion_to_cxl_candidate_bfs

CXL -> SSD, cool_cxl_candidate_bfs 和 evict_to_ssd_candidate_bfs

中间加上是否同一层的判断

(如果父节点位于 DRAM，判断逻辑限制在本层的 DRAM 子节点里) (或者不做跨层限制，

如果父节点位于 DRAM，

不管有没有子节点位于 DRAM，哪怕子节点位于 CXL，只要是 Hot, 就驱逐) 。

参考资料:

[1] https://blog.csdn.net/weixin_29215815/article/details/158186300

[2] https://mp.weixin.qq.com/s/_qEant6NCm8bKrcCWggTAQ

Phase 2

准入控制算法(全部位于 DRAM)

先验证算法 ==> 证明引入 Two-level-admission-control 比通常的基于 Page 粒度的 LRU 要更好。

构造的是后台的 Admission-control 线程。

算法的逻辑是:

涉及到的数据结构:

- (1) Count-Min-Sketch,
- (2) 访问频率直方图,
- (3) 分配 12 个 bucket(bucket 可调) 来记录页面访问频率落在 $[2^n, 2^{n+1} - 1]$ 之间的数目, 从而根据 DRAM 和 CXL 容量大小的不同比例来确定准入 DRAM Buffer Pool 的访问水位。比如在 DRAM : CXL = 1: 2 的情况下, 访问准入的阈值是 20 次访问, 而在 DRAM: CXL = 1: 4 的情况下, 访问准入的阈值是 50 次访问, 在 DRAM: CXL = 1:8 的情况下, 访问准入的阈值是 60 次访问。

(这里同时需要做一个实验, 在我们的背景下, Record 符合 Zipfian 分布, 比如说极端的页内访问倾斜, 也就是整体的数据呈现 Zipfian 分布, 同时同一个 Page 上实际访问到的 Record 一定程度上也体现了 Zipfian 分布的特点, 之前我们的实验数据是, Top 1% ~ Top 2% 最热的 Slots 贡献了高达 69.1% 的页面访问量, Top 5% 的 Slots 贡献了 81.4% 的访问量) 。【在两阶段识别完成之后, 看我们的算法准不准, 实验中查询请求的次数调大】

1. 维护一个 page_id 访问对应的 count_page_min_sketch 二维数组, 比如 count_page_min_sketch[12][1024 * 16]
2. (这里维护全局的访问频率直方图, 目的是准确识别热点信息。但 count_page_min_sketch 本身是流式的, 如果直接将每次访问动态归类到某一个 bucket 0 中, 会出现同一个 page_id 先出现在 Bucket 2, 经过一段时间的访问, 又在 Bucket 3 中统计了一次的重复统计问题。

因此为了解决 count_page_min_sketch 的流式统计和 Global Page Count Distribution 是在某一个时刻的静态状态快照的问题, 我们采用 1024 大小的去重 set, 采样记录当前访问过程中出现的 1024 个不同 page_id, 读取 count_page_min_sketch 并填写对应 Bucket 信息, 维护一个局部访问频率直方图, 用局部分布代替整体, 局部访问频率直方图在 30s (修改: 每处

理 1 万次请求)-后清空, 去重 set 也置空, 进入下一轮, 在当前时间窗口内保证没有重复统计)。

3. 桶对应应有 ~~30s~~ 的超时老化机制, ~~每 30s 会把指针向后移动~~, (修改: 每处理 1 万次请求, 进行一次采样和直方图更新) head 永远表示记录访问次数在 [1] 的数据, Aging 机制可以直接将访问计数减半, 对应只需要数组移位, Buckets 可以直接用数组表示。

count_page_min_sketch 的老化采用 $\gg 1$ 的操作, 访问计数减半。

4. 根据当前的 bucket 建立一个横坐标为 12 个不同的访问区间, 纵坐标为不同 page_id 数目的直方图, 根据容量比例找到需要进入 RecordCache 的 Bucket 对应的访问次数, 比如对应访问次数在 2^x 之上。

(前 4 步都是低开销, 不涉及高频并发争抢的操作)

5. 动态处理, 当一个 page_id 请求到来时, 如果在 count_page_min_sketch 的基础上发现当前访问次数在 2^x 之上, 则标记该页面是可以进入 DRAM 的 Candidate, 如果计算出访问频率超过符合 DRAM 缓存容量的阈值时, 进入 dram_admission_candidates 数组中进行候选者筛选。

=> 第一层筛选, 确保随着访问次数变化, 进入 DRAM 的永远是最热的数据。

=> 第二层筛选, 区分出 Hot Page 和 (Cold Page, Hot Record), 保证 RecordCache 针对倾斜访问的负载尽可能节省 DRAM 空间的使用, 提高 lookup 查询效率。

6. 对进入 dram_admission_candidates 的 page 维护每个 page_id_record_id 的访问次数统计, 从中筛选出倾斜的访问。区分出 Hot Page 和 (Cold Page, Hot Record) 的情况, Hot Record 进入 RecordCache

(这个设计的 trade-off 在于前几轮对 page 的访问流只累积在了 count_page_min_sketch 里, 对于 record 的访问是在已经判断出 page_id 之后才开始记录的, 丢失前几轮访问的信息)。

如果在第二层筛选中发现对页面的访问是均匀的, 则全页提升到 DRAM Buffer Pool, 否则只提升倾斜性的 record 到 RecordCache。

具体的实现流程:

1. 核心的结构是 Count-Min-Sketch, 二维数组大小的参数可以自己控制。
2. 访问频率直方图的表和 bucket 桶, 以及环形数组的结构
3. 第一层 Hot Page 的筛选结构
4. 把所有的 Hot Page 加入到 dram_hot_page_candidates 之后, 这里改一下原来的逻辑, 这个 dram_hot_page_candidates 里放入这样的 tuple,

dram_hot_page_candidates: <page_id, tuple<total_visit_count, page_slot_num, uniform_or_skew, std::bitset<page_slot_num> distinct_accessed_slot_set(最多一个 Page 上有 175 个 slot), std::bitset<page_slot_num> already_promoted_slot>>.(这里考虑要不要把 bitset 换成 std::atomic<uint64_t> [3]).

(这里的 `page_visit_count` 是第一层筛选时候的 Count-Min-Sketch 得到的) 是筛选时刻的值, 不是 CMS 老化时的值), 对每一个 `dram_hot_page_candidates` 做 record-level 的跟踪 (如果产生访问, 就记录到对应的 count-min-sketch 里, `uniform_or_skew` 初始值为 `false` 表示 uniform, 如果发现 Hot-Record, 则设置为 `true`, 同时增加一个 `std::bitset<page_slot_num> already_promoted_slot`, 记录当前 `page_id` 已经 Promote 到 RecordCache 的记录, 防止反复 Promote 同一个 record)。

5. 对于均匀程度的判断, 如果发现某一个 page 上对于的 `page_id_record_id` 的访问超过了当前 `page_id` 的 $x\%$, 则判定为不倾斜。 x 是后台可以配置的参数, 初始时调整为 10% (也就是单个 Slot 的访问占据了上一轮记录的 Page 访问次数的 10%), 一旦发现某条记录的访问次数超过总 Page 的 10% , 则表明 `uniform_or_skew = true`。 (修改: 这里的 x 我们可以出一个 Insight, 具体该怎么认定 Hot Page 是 uniformly Hot 还是只是由单个 Hot 撑起来的访问量, 比如均匀访问次数用 $1/\text{page_slot_num} = 0.6\%$ 左右, 从 Motivation 的实验反推)。
6. 关于 Page 是否 uniform 还是 skew 决策之后的 promote, 如果 `distinct_accessed_slot_set` 的访问达到 50% 的 `page_slot_num`, 也就是已经访问了页面上 50% 的 Slot, 还没有触发 skew 的条件, 就默认为 uniform, promote 到 DRAM BufferPool, 一旦发现有一个 record 表示 skew, 那么 `uniform_or_skew = true`, 该页面失去 Promote 到 Buffer Pool 的机会(可调)
7. 判断 Page 需要被 Promote 到 DRAM Buffer Pool 需要两个条件,
 - a. `distinct_accessed_slot_set.size() >= 50\% * page_slot_num`
 - b. 到目前为止, 没有出现这样的 slot, 达到 `count-min-sketch.page_id_record_id > total_visit_count * 10\%`。





具体实现过程中涉及的文件:

1. Count-Min-Sketch 结构, 这里 `page_id_count_min_sketch` 和 `page_id_record_id_count_min_sketch` 可以选择使用同一个父类, 不同的子类。二维数组的大小全部设置成可以调整的参数。(留老化机制接口) ✓
2. `VisitFrequencyBucket`, 数组结构, 根据情况可以设置 12 - 16 - 更多的 Bucket, 记录访问阈值在 $[2^n, 2^{(n+1)} - 1]$ 之间的值(留老化机制接口), 老化机制直接数组移位 (`buckets[i].page_in_range_count = buckets[i + 1].page_in_range_count;`) ✓
3. 去重 Set, 当访问次数累计达到 1w 次查询时, 开始采样 1024 个不同 Page 的访问记录 (`page_id, visit_count`), 当前访问记录的获得是通过, 记录 `page_id`, 读 `PageCountMinSketch` 结构获得访问次数。✓
4. `SampledPageVisitHistogram` 访问频率直方图, 提供当前可以进入 DRAM 的访问次数水位线, 水位线根据 DRAM / CXL 的比例来进行选取(`VisitHisogram` 需要联动前面的 `Page_Count_Min_Sketch`, `VisitFrequencyBucket`, 去重 Set) ✓
5. `dram_hot_page_candidates`:

`<page_id, tuple<total_visit_count, page_slot_num, uniform_or_skew, std::bitset<page_slot_num> distinct_accessed_slot_set(最多一个 Page 上有 175 个 slot), std::bitset<page_slot_num> already_promoted_slot>>`, `page_slot_num` 需要从当前 Page 的结构信息上获得。

这里描述一下 `dram_hot_page_candidates` 的生命周期:

(解决遇到的核心问题: 怎么确认所有倾斜的记录都已经 Promote 到 DRAM 里, 超过多少轮没有得到明确的 Promote 信息, 怎么设置)

【`dram_hot_page_candidates` 准入】:

准入部分的考虑, 准入的来源有两个:

(1) 流式页面访问的时候, 发现超过阈值

(2) 当前访问频率直方图建立完成, 得到阈值之后, 扫描一遍采样数据集中在当前阈值之上的 page_id, 这些 page_id 也加入 dram_hot_page_candidates. (这部分可加可不加)

【dram_hot_page_candidates 删除】:

每个 page_id 进入的时候, 封装成 HotPageCandidate, 每个 HotPageCandidate 有一个访问计数器:

per_hot_page_visit_count,

以及 per_hot_page_requests_snapshot (选取开始时候的快照)

dram_hot_page_candidates 也维护一个全局的 global_requests++ (只要有 Page 访问就统计, 不区分是不是 dram_hot_page_candidates 里的 page)

(1) 在 per-hotPageCandidate_visit_count >= 3000 次, 还没有判断出是否 skew, 则全页 Promote, 删除 HotPageCandidate

(2) 在 per-hotPageCandidate_visit_count < 3000 次, 已经判断出是 skew, 则对于 skew 的记录逐一 Promote,

如果发现 global_requests - per_hot_page_requests_snapshot > 5w, 默认不再产生 skew record 的 Promote 行为, 从 dram_hot_page_visit_count 中删除当前 HotPageCandidate。

(RecordCountMinSketch 的老化逻辑独立, 每当 dram_hot_page_candidates 的 global_requests 次数增加超过 1w, 则执行老化, 和 PageCMS 的老化保持一致)。✅

6. promote 的方法, (1) Promote_page (2) Promote_record (目前先不处理, 等到 RecordCache 全部实现完毕之后处理)

7. 后台注册这个 two-level-admission-control, 不处理分片, 全局的。

✓ 关于当前部分编写单元测试的脚本

📄 复制代码

```
g++ -std=c++17 -pthread -I. -Ishared-headers -o two-level-admission-  
test-suite/test_cms two-level-admission-test-  
suite/test_count_min_sketch.cpp  
  
# SampledVisitHistogram 用到了 gflags 库  
g++ -std=c++17 -pthread \  
-I. \  
-Ishared-headers \  
-Ibuild/vendor/gflags/include \  
-o two-level-admission-test-suite/test_sampled_visit_histogram \  
two-level-admission-test-suite/test_sampled_visit_histogram.cpp \  
backend/leanstore/Config.cpp \  
-Lbuild/vendor/gflags/lib -lgflags  
./two-level-admission-test-suite/test_sampled_visit_histogram 2>&1 |  
tee "  
pagethreshold
```

.md"

```
g++ -std=c++17 -pthread -I. -Ishared-headers -o two-level-admission-  
test-suite/test_skew_record_admission two-level-admission-test-  
suite/test_skew_record_admission.cpp
```

```
g++ -std=c++17 -pthread -I. -Ishared-headers -  
Ibuild/vendor/gflags/include -o two-level-admission-test-  
suite/test_two_level_admission_control two-level-admission-test-  
suite/test_two_level_admission_control.cpp
```

```
g++ -std=c++17 -pthread -I. -Ishared-headers -  
Ibuild/vendor/gflags/include -o two-level-admission-test-  
suite/test_large_scale_admission two-level-admission-test-  
suite/test_large_scale_admission.cpp -O3
```

```
g++ -std=c++20 -pthread \  
-I. \  
-Ishared-headers \  
-Ibuild/vendor/gflags/include \  
-Lbuild/vendor/gflags/lib \  
-o record-cache-test-suite/test-recordcache \  
record-cache-test-suite/test-recordcache.cpp \  
-lgflags
```

测试 Forward_Epoch

```
g++ -std=c++20 -pthread \  
-I. -Ishared-headers \  
-Ibuild/vendor/gflags/include \  
-Lbuild/vendor/gflags/lib \  
-o record-cache-test-suite/test-forward-epoch-thread \  
record-cache-test-suite/test-forward-epoch-thread.cpp \  
backend/leanstore/storage/record-cache/RecordCache.cpp \  
backend/leanstore/storage/record-  
cache/Forward_epoch/ForwardEpochThread.cpp \  
backend/leanstore/storage/record-  
cache/SIEVE_eviction/SIEVEEvictionThread.cpp \  
backend/leanstore/Config.cpp \  
-lgflags
```

专门用于 Debug 的编译命令

```
g++ -std=c++20 -pthread -fsanitize=address -g \  
-I. \  
-Ibackend \  
-Ishared-headers \  

```

```

-Ibuild/vendor/gflags/include \
-Lbuild/vendor/gflags/lib \
-o record-cache-test-suite/test-forward-epoch-thread \
record-cache-test-suite/test-forward-epoch-thread.cpp \
backend/leanstore/storage/record-cache/RecordCache.cpp \
backend/leanstore/storage/record-
cache/Forward_epoch/ForwardEpochThread.cpp \
backend/leanstore/storage/record-
cache/SIEVE_eviction/SIEVEEvictionThread.cpp \
backend/leanstore/storage/record-cache/Promote/PromoteThread.cpp \
backend/leanstore/utils/RandomGenerator.cpp \
backend/leanstore/Config.cpp \
-lgflags

g++ -std=c++20 -pthread -fsanitize=address -g \
-I. -Ishared-headers \
-Ibuild/vendor/gflags/include \
-Lbuild/vendor/gflags/lib \
-o record-cache-test-suite/test_sieve_eviction \
record-cache-test-suite/test_sieve_eviction.cpp \
backend/leanstore/storage/record-cache/RecordCache.cpp \
backend/leanstore/storage/record-
cache/Forward_epoch/ForwardEpochThread.cpp \
backend/leanstore/storage/record-
cache/SIEVE_eviction/SIEVEEvictionThread.cpp \
backend/leanstore/storage/record-cache/Promote/PromoteThread.cpp \
backend/leanstore/Config.cpp \
-lgflags

# 测试 SIEVE Eviction logic
g++ -std=c++20 -pthread -fsanitize=address -g \
-I. \
-Ibackend \
-Ishared-headers \
-Ibuild/vendor/gflags/include \
-Lbuild/vendor/gflags/lib \
-o record-cache-test-suite/test_sieve_eviction \
record-cache-test-suite/test_sieve_eviction.cpp \
backend/leanstore/storage/record-cache/RecordCache.cpp \
backend/leanstore/storage/record-
cache/Forward_epoch/ForwardEpochThread.cpp \
backend/leanstore/storage/record-
cache/SIEVE_eviction/SIEVEEvictionThread.cpp \
backend/leanstore/storage/record-cache/Promote/PromoteThread.cpp \

```



```
backend/leanstore/utils/RandomGenerator.cpp \  
backend/leanstore/Config.cpp \  
-lgflags  
  
# 测试 Promote Thread logic  
# test-real  
g++ -std=c++20 -pthread -fsanitize=address -g \  
-I. \  
-Ibackend \  
-Ishared-headers \  
-Ibuild/vendor/gflags/include \  
-Lbuild/vendor/gflags/lib \  
-o record-cache-test-suite/test-promote-real-thread \  
record-cache-test-suite/test-promote-real-thread.cpp \  
backend/leanstore/storage/record-cache/RecordCache.cpp \  
backend/leanstore/storage/record-  
cache/Forward_epoch/ForwardEpochThread.cpp \  
backend/leanstore/storage/record-  
cache/SIEVE_eviction/SIEVEEvictionThread.cpp \  
backend/leanstore/storage/record-cache/Promote/PromoteThread.cpp \  
backend/leanstore/Config.cpp \  

```

目前维护 VisitFrequencyBucket 的部分已经完成，单元测试结果符合预期。

测试：模拟真实准入控制场景

模拟 100 个页面的访问...

访问频率分布直方图：

Bucket	访问范围	页面数量	累计数量(从此bucket及以上)
Bucket_0	[1, 1]	10	100
Bucket_1	[2, 3]	20	90
Bucket_2	[4, 7]	20	70
Bucket_3	[8, 15]	16	50
Bucket_4	[16, 31]	8	34
Bucket_5	[32, 63]	6	26
Bucket_6	[64, 127]	6	20
Bucket_7	[128, 255]	5	14
Bucket_8	[256, 511]	4	9
Bucket_9	[512, 1023]	1	5
Bucket_10	[1024, 2047]	2	4
Bucket_11	[2048, 4095]	1	2
Bucket_12	[4096, 8191]	1	1

===== 准入阈值分析 =====

场景：DRAM:CXL = 1:2 (33% DRAM)

DRAM 容量可容纳页面数：33/100

-> 准入阈值：访问次数 >= 32 的页面 (Bucket_5 及以上)

-> 可准入页面数：26

-> 容量利用率：78.7879%

场景：DRAM:CXL = 1:4 (20% DRAM)

DRAM 容量可容纳页面数：20/100

-> 准入阈值：访问次数 >= 64 的页面 (Bucket_6 及以上)

-> 可准入页面数：20

-> 容量利用率：100%

场景：DRAM:CXL = 1:8 (11% DRAM)

DRAM 容量可容纳页面数：11/100

-> 准入阈值：访问次数 >= 256 的页面 (Bucket_8 及以上)

-> 可准入页面数：9

-> 容量利用率：81.8182%

[✓] 模拟真实准入控制场景 通过！

Phase 3(RecordCache-ReadOnly)

加速 Lookup, 将 Update 操作转化为对 Cache 的 Invalidation, 利用 Epoch 机制实现无锁化的并发读, 采用 SIEVE 算法进行低开销的驱逐。RecordCache 是全局的, 不搞分片。

1. RecordCacheEntry 结构和字段的封装 
2. Slab Allocator 内存分配器(直接选择一个外部库, 能够支持 2 MB 申请大页, 8 Byte 步长划分不同 size 的 free_list)

(这里还是没有避免重复造轮子,

目前加上了 C++17 特有的 `std::pmr`(Polymorphic memory resource) 作为回退的情况,

核心是重写了 `do_allocate`, `do_deallocate`, `do_is_equal`,

特点是接管 `BufferManager` 里 `mmap` 出来的 `record_cache_ptr`, 然后自定义不同 `SlabClass` 的懒加载和切块,

超过定义情况时, `pmr` 自动回退到 `new_and_delete` 的 `new/delete` 方法)。✅

```
● (base) [root@GNR-91 cxi-recordcache-dev]# ./record-cache-test-suite/test-recordcache-slab-allocator-2

=====
RecordCacheSlabAllocator Verification Test Suite
(Proving SlabAllocator is actually being used)
=====

TEST 1: Pointer Range Verification
=====
[INFO] Allocating entries via SlabAllocator and verifying pointers are inside mmap pool
[INFO] mmap pool range: [140000162594816, 140000183566336)
allocate( 24 B) -> ptr=0x7f5453f5c000 [IN POOL ✓]
allocate( 32 B) -> ptr=0x7f545415c000 [IN POOL ✓]
allocate( 64 B) -> ptr=0x7f545435c000 [IN POOL ✓]
allocate( 256 B) -> ptr=0x7f545455c000 [IN POOL ✓]
allocate( 1024 B) -> ptr=0x7f545475c000 [IN POOL ✓]
allocate( 4096 B) -> ptr=0x7f545495c000 [IN POOL ✓]
allocate( 16384 B) -> ptr=0x7f5454b5c000 [IN POOL ✓]
[PASS] All allocated pointers are inside mmap pool -> SlabAllocator is working!

=====
TEST 2: Pool Exhaustion Verification
=====
[INFO] Using a small pool (4 MB) and allocating until exhaustion
[INFO] SlabAllocator must throw std::bad_alloc when pool is full
[INFO] (System malloc would never fail due to our mmap pool size)
[INFO] Pool size: 4 MB, Allocation size: 256 B
[INFO] Theoretical max allocations: 16384
[INFO] Actually allocated: 16384 blocks before exhaustion
[PASS] Pool exhausted at 16384 blocks (≤ 16384) -> SlabAllocator boundary enforced!
[PASS] After deallocate, re-allocation succeeded -> free_list recycle works!

=====
TEST 3: Slab Alignment and Step Verification
=====
[INFO] Allocating multiple blocks of same size, checking pointer spacing == block_size
request= 24 B -> aligned block_size= 24 B, pointer spacing consistent ✓
request= 25 B -> aligned block_size= 32 B, pointer spacing consistent ✓
request= 30 B -> aligned block_size= 32 B, pointer spacing consistent ✓
request= 32 B -> aligned block_size= 32 B, pointer spacing consistent ✓
request= 100 B -> aligned block_size= 104 B, pointer spacing consistent ✓
request= 256 B -> aligned block_size= 256 B, pointer spacing consistent ✓
[PASS] All size classes show correct 8-byte step alignment -> SlabAllocator slab layout verified!
```

```

=====
TEST 4: Concurrent Allocation - No Duplicate Pointers
=====
[INFO] 64 threads allocating simultaneously, all pointers must be unique and in-pool
Extensions (0%): -1 requires update, 2 require reload, 1 requires allocating
[INFO] Phase 2 complete: uniqueness check done
[INFO] Total allocations:      64000
[INFO] Out-of-pool pointers: 0
[INFO] Duplicate pointers: 0
[INFO] Phase 3 complete: all memory deallocated
[PASS] All 64000 pointers are unique and inside mmap pool -> Thread-safe SlabAllocator confirmed!

=====
TEST 5: Deallocate Recycle (free_list Verification)
=====
[INFO] Allocate N blocks, deallocate them, reallocate - new ptrs must match old ptrs
[INFO] First round: allocated 10 blocks
[INFO] Deallocated all 10 blocks back to free_list
[INFO] Second round: allocated 10 blocks
First round addresses: 7f545495c000 7f545495c080 7f545495c100 7f545495c180 7f545495c200 7f545495c280 7f545495c3
00 7f545495c380 7f545495c400 7f545495c480
Second round addresses: 7f545495c000 7f545495c080 7f545495c100 7f545495c180 7f545495c200 7f545495c280 7f545495c3
00 7f545495c380 7f545495c400 7f545495c480
[PASS] Second round pointers exactly match first round -> free_list LIFO recycle confirmed!

=====
TEST 6: Oversized Allocation Fallback Verification
=====
[INFO] Allocating > 16 KB should fallback to system new/delete, NOT from mmap pool
allocate( 16384 B): MAX_BLOCK_SIZE (16 KB) -> in pool -> IN POOL ✓
allocate( 16392 B): MAX_BLOCK_SIZE + 8 B -> fallback to system heap -> SYSTEM HEAP ✓
allocate( 65536 B): 64 KB -> fallback to system heap -> SYSTEM HEAP ✓
allocate( 1048576 B): 1 MB -> fallback to system heap -> SYSTEM HEAP ✓
[PASS] Oversized fallback path verified -> do_allocate boundary logic correct!

=====
TEST 7: RecordCache Integration (End-to-End)
=====
[INFO] Allocate entries via SlabAllocator, insert into RecordCache, verify all ptrs in pool
[INFO] Inserted 100 entries (all from mmap pool)
[INFO] Found 100/100 entries, all pointers match SlabAllocator allocations
[PASS] End-to-end: SlabAllocator -> RecordCache -> verified all pointers in mmap pool!

=====
All 7 Tests Passed - SlabAllocator is genuinely working!
=====

```

3. RecordCache 结构和字段的封装(Hash 表)

(1) 高并发的哈希表, 链地址法  (2) 带有一个 Slab Allocator, 使用 2 的 RecordCacheSlabAllocator 

4 Epoch 机制和 Forward_epoch

- (1) Forward_epoch 线程负责推进 global_epoch
- (2) 线程局部变量 ThreadEpochState: {std::atomic<bool> active; std::atomic<uint64_t> current_epoch;}
- (3) Worker 线程启动时绑定线程局部变量
- (4) Invalidation Queue 并发失效队列(Multi-Producer-Single-Consumer)
- (5) Forward_epoch 线程的核心执行逻辑

Forward_epoch 线程的执行流程:

1. 周期性更新全局变量 global_epoch, 执行推进。
2. Invalidation_Queue 中一旦有内容, 根据当前 Invalidation_Queue 中的 {Invalid_RecordCacheEntry, update_epoch} 读出 update_epoch,

轮询所有当前分片对应的 {epoch_active_state, current_epoch}, 检查所有小于 update_epoch 的 epoch 状态, 如果还有 active 的情况, 说明逻辑删除的 RecordCacheEntry 仍然在被 Lookup 线程使用, 不能执行物理删除。否则执行物理删除。

3. 物理删除涉及两个操作:

- a. 获取 RecordCache 的 bucket 锁, 执行 hashtable.remove(key, entry_pointer), 从 RecordCache 中删除 key。
- b. 同时把 entry_pointer 对应的 RecordCacheEntry Type 字段设置为 100, 表示已经从哈希表中移除。

目前分为三个文件:

EpochManager , InvalidationQueue , ForwardEpochThread 

(待通过 test-forward-epoch-thread.cpp 的测试)

5 Eviction 机制和 SIEVE Eviction 线程

由 Eviction 来做 Slab Allocator 的内存释放。

SIEVE Eviction 线程的执行流程:

1. 从 FIFO 队列中取出 entry_pointer
2. 检查 entry_pointer 的 Type 信息。
 - a. Type: 000, hand 指针对应 visited = 0,

此时是由于长期无访问导致的驱逐, 获取 HashTable 的 Bucket 锁, 在拿到锁之后 Double-Check Type 字段, 在 Type = 000 的情况下, 执行 hashtable.remove(key, entry_pointer), 调用 Slab Allocator 执行内存释放。

- b. Type: 100,

此时是由于 Update 导致记录失效, 直接调用 Slab Allocator 执行内存释放。

- c. Type: 011/111: 当前记录属于中间状态, 将 entry_pointer 重新插入 SIEVE FIFO 队列的尾部。

目前涉及的文件: (1) SIEVE 核心算法  (2) SIEVE 的 FIFO 队列  (3) SIEVEvictionThread 

6 Promote Thread

Promote 线程的执行流程。负责和 CXL Buffer Pool 的交互过程。

Promote 线程的执行流程:(Promote 线程完全不负责释放物理内存资源的工作, 只有 Eviction 线程负责这个工作)

1. 接收到 Lookup 线程的唤醒信号, 和对应的 Record 信息。
2. 在 RecordCache 中查找是否有对应的项。

- a. 只有在 RecordCacheType 为 RemovedFromHashTableWaitForMemoryReallocation 或者 Not Found 的情况下才执行接下来的操作。
- b. 其他情况直接返回

获取当前 RecordCache 的 Shard 分片锁, Promote 线程从 Slab Allocator 中申请对应 RecordCacheEntry size 的空闲内存, 初始化时 Type 字段设置为 111, 表示由 Promote 线程置位控制, 插入当前 RecordCacheEntry 进 RecordCache。插入完成之后解锁。将新的 RecordCacheEntry 记录加入 sieve_fifo_queue。

3. 无锁情况下, Promote 线程在 CXL BufferPool 中读取记录, 写到自己的线程局部缓存中。
4. 获取 HashTable bucket 锁, 先确认 Type 字段仍然是 111, 没有发生 Promote 线程读(A) -> Update 线程改 (B) -> Promote 线程写(A) 这样的并发问题。
 - a. 如果 Type 字段被 Update 线程失效为 011, 则由于 Promote 线程已经持有 bucket 锁, 在当前持锁状态将该 RecordCacheEntry 从 HashTable 中执行 HashTable.remove(key, entry_pointer), 修改 RecordCacheEntry 字段状态为 100, 表示已经从哈希表中删除, 等待 Eviction 线程执行物理内存释放。
 - b. 如果 Type 字段仍然是 111, (double-check) 通过, 将数据从局部缓存中拷贝到对应字段, 新 Record 准入操作完成, Type 字段变成 000。



小阶段测试:

三个测试文件测试 3 个后台线程:

- test_sieve_eviction.cpp (需要去掉后台 70% 水位的逻辑)

- test-forward-epoch-thread.cpp

- test-promote-real-thead.cpp(打开后台 70% 水位的逻辑, 不然 Promote 会很快被驱逐)




复制代码

```
# sieve 驱逐逻辑的编译
g++ -std=c++20 -pthread -fsanitize=address -g -I. -Ibackend
-Ishared-headers -Ibuild/vendor/gflags/include -
Lbuild/vendor/gflags/lib -o record-cache-test-
suit/test_sieve_eviction record-cache-test-
suit/test_sieve_eviction.cpp backend/leanstore/storage/record-
cache/RecordCache.cpp backend/leanstore/storage/record-
cache/Forward_epoch/ForwardEpochThread.cpp
backend/leanstore/storage/record-
```

```

cache/SIEVE_eviction/SIEVEEvictionThread.cpp
backend/leanstore/storage/record-cache/Promote/PromoteThread.cpp
backend/leanstore/utils/RandomGenerator.cpp
backend/leanstore/Config.cpp      -lgflags

# forward-epoch 逻辑的编译
g++ -std=c++20 -pthread -fsanitize=address -g \
    -I. \
    -Ibackend \
    -Ishared-headers \
    -Ibuild/vendor/gflags/include \
    -Lbuild/vendor/gflags/lib \
    -o record-cache-test-suite/test-forward-epoch-thread \
    record-cache-test-suite/test-forward-epoch-thread.cpp \
    backend/leanstore/storage/record-cache/RecordCache.cpp \
    backend/leanstore/storage/record-
cache/Forward_epoch/ForwardEpochThread.cpp \
    backend/leanstore/storage/record-
cache/SIEVE_eviction/SIEVEEvictionThread.cpp \
    backend/leanstore/storage/record-cache/Promote/PromoteThread.cpp \
    backend/leanstore/utils/RandomGenerator.cpp \
    backend/leanstore/Config.cpp \
    -lgflags

# promote-thread 逻辑的编译
g++ -std=c++20 -pthread -fsanitize=address -g \
    -I. \
    -Ibackend \
    -Ishared-headers \
    -Ibuild/vendor/gflags/include \
    -Lbuild/vendor/gflags/lib \
    -o record-cache-test-suite/test-promote-real-thread \
    record-cache-test-suite/test-promote-real-thread.cpp \
    backend/leanstore/storage/record-cache/RecordCache.cpp \
    backend/leanstore/storage/record-
cache/Forward_epoch/ForwardEpochThread.cpp \
    backend/leanstore/storage/record-
cache/SIEVE_eviction/SIEVEEvictionThread.cpp \
    backend/leanstore/storage/record-cache/Promote/PromoteThread.cpp \
    backend/leanstore/Config.cpp \
    backend/leanstore/utils/RandomGenerator.cpp \
    -lgflags

```

7 WorkerThread lookup logic Intergration

Lookup 线程的执行流程:

1. 读取 Global_epoch, 记录下当前线程局部变量状态 {active, current_epoch = lookup_global_epoch}
2. 判断 RecordCache 中是否有当前记录, Hash(key)

根据 Hash(key) 的返回值, 有如下情况:

a. Type = 000, Hit, 直接返回

b. Type = 011, 逻辑删除, 读 CXL Buffer Pool, 将对应 Count-Min-Sketch 的位置访问次数 +1, 判断是否超过阈值, 超过则唤醒 Promote 线程。给 Promote 线程发送 CXL Buffer Pool 中 RecordCacheEntry 地址的相关信息, 比如 (cxl_address, offset), 方便 Promote 线程的直接内存访问。

c. Type = 111, 已经有线程在处理 Promote, 读 CXL BufferPool, 将对应 Count-Min-Sketch 的位置访问次数 +1。

d. Not Found, 读 CXL Buffer Pool, 将对应 Count-Min-Sketch 的位置访问次数 +1, 判断是否超过阈值, 超过则唤醒 Promote 线程。给 Promote 线程发送 CXL Buffer Pool 中 RecordCacheEntry 地址的相关信息, 比如 (cxl_address, offset), 方便 Promote 线程的直接内存访问。

3. 退出之前将当前线程局部变量的 active 设置为 false。

8 WorkerThread update logic Intergration

Update 线程的执行流程:

1. 读取 Global_epoch, 记录下当前线程的状态 {active, current_epoch = update_global_epoch}
2. 判断 RecordCache 中是否有当前记录, Hash(key)

根据 Hash(key) 的返回值, 有如下情况:

a. Type = 000, 立刻调用 CAS(Compare-and-Set), 将 Hash(key) 对应的 Type 字段设置为 011, 标记为逻辑删除。

b. Type = 111, 不能释放, 因为 Promote 线程还在使用。这里仅设置 Type 字段为 011, 由 Promote 线程完成释放。

c. Type = 011, 已经有线程标记过逻辑删除, 只需要更新 CXL BufferPool。

d. Not Found, 不需要处理 RecordCache 相关逻辑, 只需要更新 CXL BufferPool。

3. 对于 2.1 中的情况, Update 线程负责将对应逻辑删除的记录放入 Invalidation_Queue 其中失效队列是分片的, 可以实现成 FIFO 队列, 将 {Invalidation_RecordCacheEntry, update_global_epoch} 放入 Invalidation_Queue。
4. 退出之前将当前线程局部变量的 active 设置为 false。

9 Two-Level Admission Promote Logic

10 WorkerThread register

发现遇到一个分片的问题，

我之前设计的两层准入算法，目前是根据 cxi partitions 来的(相当于只看 page_id 路由到哪些 partitions)，然后后台的两层准入线程就负责线程所属的 partitions 的 Promote full page or Promote to RecordCache 的逻辑。

但是 RecordCache 这里的分片是根据表id 来的，相当于注册的时候有一个 data_structure_id，不同的表对应不同的 data_structure_id，所以这里分片是逻辑上属于同一张表的就分在一个分片里。

全部不搞 Parititon.(Partition 容易造成 Workload Skew 的问题，更加严重)。

遗留优化点:

1. resolveSwip 那里直接从上层拿到是索引页的信息，少一次 CXL 分配，回收，DRAM 分配，拷贝的开销。
2. DramPageEviction 的时候不区分孩子节点是不是同层，只要有 COOL 的孩子节点，就放入队列。
3. Two-level-admission-control 这里的准入配置参数: (老化参数，数据规模参数，skew 阈值判断)
4. double-check 一下这里 cxi 的 free_list 分配是不是确实走的是 cxi mmap 出来的那块内存。
5. RecordCacheSlabAllocator 这里用了一个扩展功能 std::pmr，本意是为了支持 CXL 上的 std::vector 分配的写法，这里其实不需要，有点过度设计，如果性能在这里拖慢了的话，彻底改掉这部分内容。
6. RecordCache 的 sieve eviction 缺少一个唤醒的水位（合适的情况是当 RecordCache 的可用容量小于某个值）